Robotics and Autonomous Systems 191 (2025) 105000

Contents lists available at ScienceDirect

Robotics and Autonomous Systems

journal homepage: www.elsevier.com/locate/robot

Multi-Agent Pickup and Delivery with external agents

Luca Bonalumi[1], Benedetta Flammini [*,1], Davide Azzalini, Francesco Amigoni

Politecnico di Milano, Dipartimento di Elettronica, Informazione e Bioingegneria, Via Ponzio 34/5 Milan, 20133, Italy

ARTICLE INFO

Keywords:
Multi-Agent Pickup and Delivery
Multi-robot systems
Modeling other agents

ABSTRACT

In a Multi-Agent Pickup and Delivery (MAPD) problem, a team of agents moving in an environment must complete pickup and delivery tasks while avoiding collisions. These tasks appear and are assigned dynamically at runtime, assuming that the environment is static, except for the movements of the agents. This paper introduces a novel MAPD formulation, where *team agents* must solve their MAPD problem in an environment populated by moving *external agents*. Team agents do not communicate nor collaborate with external agents and cannot interfere with external agents' goals and paths, which are unknown. Thus, team agents are in charge of preventing collisions with external agents while fulfilling their pickup and delivery tasks. To address this challenge, we propose an approach where team agents build behavioral models of external agents, and then use these models to anticipate potential conflicts with external agents and plan paths accordingly. We employ two modeling approaches: (1) a probabilistic occupancy model that estimates the likelihood of each location in the environment being occupied by external agents and (2) a Markovian model that captures how external agents transition between locations in the environment. The proposed approach is evaluated under different observational assumptions for team agents, demonstrating that leveraging knowledge of external agents' behavior reduces collision events and task completion times for team agents.

1. Introduction

One fundamental challenge in the domain of multi-agent systems that operate in physical environments is the efficient coordination of the movements of the agents in a shared space [1]. Among the many attempts to address this challenge, one has recently received a lot of attention: Multi-Agent Pickup and Delivery [2]. The Multi-Agent Pickup and Delivery (MAPD) problem consists of finding collision-free paths for a team of agents executing incoming tasks by moving from their starting locations to the delivery locations, passing through the pickup locations. MAPD problems have an online nature, since new tasks can be added at any time, and agents continuously assign and complete them. For this reason, MAPD can be considered a combination of assigning tasks and finding collision-free paths for a team of agents. The MAPD problem is currently very popular within both academia and industry due to its several real-world applications. The most relevant from an industrial point of view is that of automated warehouses [3]. Beyond logistics, other MAPD applications are in aircraft-towing vehicles [4], office robots [5], and automated control of non-player characters in video games [6].

Usually, the agents addressing a MAPD problem are considered to be the only moving entities in the environment, and several algorithms have been proposed to plan coordinated paths that avoid collisions between them [7]. However, in some application settings, other moving agents could be present in the environment, either humans or robots [8]. For example, in a warehouse, cleaning robots can operate in the same environment as logistics robots, but the two groups of agents can belong to different companies and operate independently (Fig. 1). Another application example can be found in the context of a supermarket with aisles dedicated to groceries, clothing, and electronics. Consider a scenario where the store does not have its own fleet of robots but instead contracts individual pieces of work to separate companies. One company might be hired to deploy robots for inventory management, while another company might be hired to manage cleaning jobs, such as picking up trash from the floor to deliver it in specific bins. Although these robots work in the same environment, they do not belong to a single team (e.g., they do not necessarily share information) because each group is only concerned with its specific job.

In this paper, we consider a version of the MAPD problem in a *dynamic* environment, in which other external moving agents are present, beyond those belonging to the team and performing MAPD. Thus, we denote the proposed MAPD variant as *MAPD with external agents* (*MAPD-EA*). We assume that *team agents* cannot communicate with the *external agents* and cannot interfere with their goals and paths,

* Corresponding author.
E-mail address: benedetta.flammini@polimi.it (B. Flammini).
[1] Authors contributed equally.

https://doi.org/10.1016/j.robot.2025.105000
Received 24 December 2024; Received in revised form 5 March 2025; Accepted 26 March 2025
Available online 4 April 2025
0921-8890/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

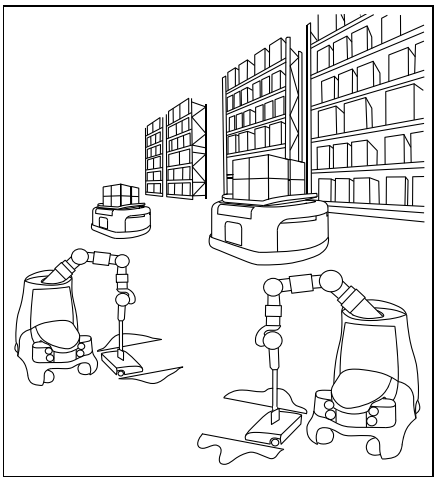


Fig. 1. An application scenario for our MAPD problem: a team of cleaning robots and a team of warehouse robots working in a shared environment.

which are unknown and cannot be modified. As a result, collisions between team and external agents may happen. When there is a potential collision, team agents have to reactively avoid the collision and replan their paths. We thus consider an asymmetrical setting in which external agents are assumed to be unaware of the presence of team agents and do nothing to avoid collisions with them.

Frequent replans can slow down the completion of tasks, implying that team agents could need more time to solve their MAPD problem. Besides, continuously computing new plans is not only time-consuming, but sometimes replanning at high frequency could be even impossible due to computing and communication limitations (e.g., when planning is not performed on board the robots but by a ground station). To overcome this issue, we propose that team agents *model* the behavior of external agents, and then use this information to plan paths that attempt to avoid collisions. Specifically, we propose two different modeling strategies: (1) a probabilistic occupancy model, partially introduced in our previous work [8], that models the probability that each location of the environment is occupied by external agents according to their past behavior, and (2) a Markovian model, that is able to capture how external agents transition between the locations of the environment based on their past behavior. Exploiting such models, team agents can plan paths that are both short and with low collision probability.

In particular, we propose a solving algorithm, called Token Passing with Collision Avoidance and Replanning + Model (TP-CA-M), that incorporates the knowledge about external agents in the planning phase of team agents by extending Token Passing [2], a common MAPD algorithm. We consider different capabilities of team agents to observe the external agents' behavior: in one case, we suppose that team agents have full knowledge of external agents' past behavior, while, in the other case, team agents have to incrementally model external agents relying only on direct observations.

We test our methods in different environments, showing that incorporating external agents' modeling effectively reduces collisions between team and external agents, particularly in environments that force structured movement patterns. Markovian models successfully capture external agents' behavior, enabling team agents to decrease the number of collisions and maintain steady task execution times, even without requiring an initial observation phase. Additionally, the reduced need for replanning offsets the extra computational cost of consulting the model, even accelerating performance in some configurations.

The main contributions of this paper are:

- The introduction and formalization of a new MAPD variant, called MAPD-EA, in which a team of agents has to solve their MAPD problem without communicating or interfering with external agents moving in the same environment.
- A solving algorithm for the MAPD-EA problem, called TP-CA-M, in which team agents plan their paths by accounting for the expected behavior of external agents.
- The testing of TP-CA-M with two different models that describe the behavior of external agents (a probabilistic occupancy model and a Markovian model), and with different observational capabilities of team agents.

This paper is organized as follows. In Section 2 we provide some background on MAPD and in Section 3 we discuss related work. Section 4 formalizes the problem we address, while Section 5 describes the proposed solving algorithm. In Sections 6 and 7 we present the two models that we developed. Sections 8 and 9 are dedicated to the description of two different experimental settings and the results obtained. Section 10 presents a broad analysis of the proposed approach and Section 11 concludes the paper.

2. Background

In this section, we review MAPD and the Token Passing algorithm, which is the MAPD solver on which our proposed solution is built.

2.1. MAPD

The basic problem formulation for coordinated path planning of a team of agents is probably that of Multi-Agent Path Finding (MAPF) [9], in which a set of agents plan collision-free paths to reach preassigned goal locations (one for each agent) starting from their current locations. However, in many real-world applications, such as automated warehouses, agents are required to carry out a stream of incoming tasks, rather than a one-shot path-finding problem. For this reason, an online extension to the MAPF framework [10] has been proposed, called Multi-Agent Pickup and Delivery, or MAPD [2].

The objective of MAPD consists of coordinating a team of agents, by assigning them to the incoming tasks and ensuring no collision happens when fulfilling the tasks. The goal is to optimize the assignment of tasks to agents, as well as the resulting path planning, so that the execution time of the tasks is minimized.

More formally, an instance of the MAPD problem consists of k agents $\mathcal{A} = \{a_1, a_2, \dots, a_k\}$, an undirected connected graph $G = (V, E)$ whose vertices in V represent the locations of the environment and edges in E represent the direct connections between adjacent (neighboring) vertices, and a task set $\mathcal{T}$ containing the tasks that have not yet been executed. Each task $\tau_j \in \mathcal{T}$ is characterized by a pickup location $s_j \in V$ and a delivery location $g_j \in V$. A task is considered to be completed after the agent in charge to fulfill it reaches g_j passing through s_j. An agent that is not currently executing any task is called *free* and it can be assigned to any task in $\mathcal{T}$, otherwise, if the agent has an assigned task it is called *occupied*. An occupied agent becomes free after it reaches the delivery location g_j of the task it is currently executing. Similarly, a free agent becomes occupied when it is assigned an available task in $\mathcal{T}$.

The formulation makes some assumptions: time is discrete, agents can perform a single *action* at each time step, and actions take exactly one time step to be executed (namely, the cost of each action is unitary). Thus, agents are located at one of the vertices in V at each time step. Actions are defined via a function $\alpha : V \to V$, so that $\alpha(v) = v'$ carries the meaning that an agent performing the action α in vertex v will move to vertex v' at the following time step. An agent can choose to stay in its current vertex by performing a *wait* action (namely, an action $\alpha(v) = v$), otherwise, it can move from its current vertex to a neighboring one by performing a *move* action (namely, an action $\alpha(v) = v'$ such that $(v, v') \in E$).

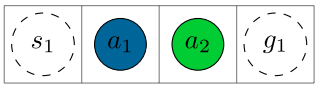


Fig. 2. An example of an unsolvable MAPD instance. Both agents are unable to complete the task: agent a_1 can reach the pickup location s_1, but not the delivery location g_1, while agent a_2 is not even able to reach the pickup location s_1.

When an agent a_i is assigned a task τ_j, the agent plans a *path* π_i, namely a sequence of locations $\pi_i = (\ell_1, \ell_2, \dots, \ell_n)$, where $\ell_t \in V$ (also denoted as $\pi_i[t] \in V$) is the location that a_i occupies at time step t. Given ℓ_t, the next location in a path π_i is $\ell_{t+1} = \alpha(\ell_t)$, where α is the action (move or wait) performed by a_i at t. A path for agent a_i starts from its starting location (i.e., $\ell_1 = \pi_i[1]$ is the starting location of a_i) and reaches the delivery location of its assigned task τ_j after n time steps (namely, $\ell_n = \pi_i[n] = g_j$), passing through the pickup location (namely, $\ell_t = \pi_i[t] = s_j$ for some $t \in [1, 2, \dots, n]$). A valid solution to a MAPD problem consists of a set of single-agent paths that have no conflicts (also called collisions) and such that all the tasks have been completed.

Two types of conflicts are considered. A *vertex conflict* occurs when agents are planned to occupy the same vertex at the same time step. Formally, $\exists t$ such that $\pi_i[t] = \pi_j[t]$ for some pair of agents a_i and a_j. A *swapping conflict* occurs when two agents swap vertices at the same time. Formally, $\exists t$ such that $\pi_i[t+1] = \pi_j[t]$ and $\pi_j[t+1] = \pi_i[t]$ for some pair of agents a_i and a_j.

Intuitively, the quality of a solution is linked to the time required to complete all tasks. A common metric is the *service time*, which is a measure of the average number of time steps required to complete a task since its appearance in the task set $\mathcal{T}$. If the number of tasks is finite, the *makespan* can be adopted as well, which is the number of time steps required to complete all tasks.

2.2. Well-formed MAPD instances

In principle, not every MAPD instance is solvable. For example, Fig. 2 shows a simple MAPD instance with two free agents and a single task, in which none of the agents is able to plan a collision-free path that reaches g_1 by passing through s_1.

The work of [2] defines a subclass of MAPD instances, called *well-formed*, for which the existence of a solution is guaranteed. The intuition is that agents are allowed to idle only at specific parking locations, called *non-task endpoints*, where they cannot block other agents.

Formally, let V_{ep} be the set of endpoints of a MAPD instance, containing all the initial locations of the agents, all the pickup and delivery locations of the tasks, and the designated parking locations. Let $V_{tsk} \subseteq V_{ep}$ denote the set of all possible pickup and delivery locations of tasks, called the set of *task endpoints*. The set $V_{ep} \setminus V_{tsk}$ is called the set of non-task endpoints.

A MAPD instance is well-formed if and only if:

(a) The number of tasks is finite.
(b) There are no fewer non-task endpoints than the number of agents, $|V_{ep} \setminus V_{tsk}| \geq k$.
(c) For any two endpoints, there exists a path between them that traverses no other endpoint in V_{ep}.

The intuitive idea behind the proof (reported in [2]) of the statement that well-formedness implies the existence of a solution is that, under these assumptions, there always exists a trivial solution that enables an agent to travel between any two endpoints, while other agents wait for the completion of tasks.

Fig. 3 illustrates three different MAPD instances. The one in Fig. 3(a) is well formed. The MAPD instance in Fig. 3(b) is not well-formed because there are fewer non-task endpoints than agents. The MAPD instance in Fig. 3(c) is not well-formed because, for instance, any path between endpoints e_3 and e_2 necessarily traverses endpoint e_1.

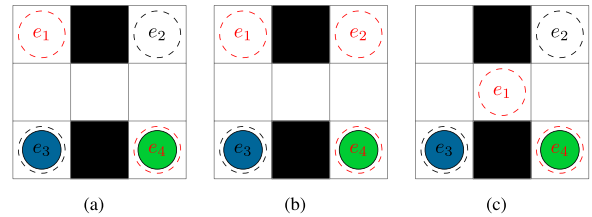


Fig. 3. Three MAPD instances. Black cells are obstacles. Blue and green circles are the initial locations of agents. Red dashed circles are task endpoints. Black dashed circles are non-task endpoints. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Algorithm 1 Token Passing

1: /* *system executes now* */
2: Initialize *token* with trivial path $(loc(a_i))$ for each agent a_i ($loc(a_i)$ is the current location of a_i)
3: **while** true **do**
4: Add all new tasks, if any, to the tasks set $\mathcal{T}$
5: **while** agent a_i exists that requests token **do**
6: /* *system sends token to* a_i — a_i *executes now* */
7: $\mathcal{T}' \leftarrow \{\tau_j \in \mathcal{T} | \text{no path in } token \text{ ends in } s_j \text{ or } g_j\}$
8: **if** $\mathcal{T}' \neq \emptyset$ **then**
9: $\tau \leftarrow \arg\min_{\tau_j \in \mathcal{T}'} h\left(loc(a_i), s_j\right)$
10: Assign a_i to τ
11: Remove τ from $\mathcal{T}$
12: Update a_i path in *token* with $PP(a_i, \tau, token)$
13: **else if** no $\tau_j \in \mathcal{T}$ exists with $g_j = loc(a_i)$ **then**
14: Update a_i path in *token* with $(loc(a_i))$
15: **else**
16: Update a_i path in *token* with $PP2(a_i, token)$
17: **end if**
18: /* a_i *returns token to system* */
19: /* *system executes now* */
20: **end while**
21: Agents move along paths in *token* for one time step
22: /* *system advances to the next time step* */
23: **end while**

2.3. MAPD solving algorithms

Optimally solving a MAPD problem requires finding a task assignment and a set of conflict-free paths that attain the shortest possible service time or makespan. Given the dynamic nature of tasks, a MAPD solver is necessarily an online algorithm, and, therefore, the task assignment is guided by some heuristics. Optimally solving a MAPD problem is challenging since it is a generalization of MAPF, which is NP-hard [11,12].

In [2], some algorithms that are able to solve any well-formed MAPD instance are presented. Here, we discuss *Token Passing* (TP) because it is the algorithm on which we base our proposed approach.

The idea behind the Token Passing algorithm is that agents plan their paths one after the other avoiding collisions with the agents that have already planned their paths. The coordination between agents relies on a synchronized shared block of memory called *token*, which is passed sequentially among agents that request it. The token contains information on the already planned paths for the agents, the task set, and the assignment of tasks. The version of TP we present here assumes the presence of a central system dealing with the token-passing mechanism; however, it can be easily extended to a fully distributed MAPD solver.

Algorithm 1 shows the pseudo-code of TP, where $loc(a_i)$ denotes the current location of agent a_i. The system starts by initializing the token with the trivial paths where each agent rests in its initial location (Line 2). At each time step, the system adds new available tasks (if any) to

the task set $\mathcal{T}$ (Line 4). The system passes the token to each of the free agents that have requested it (Lines 5–6).

The agent with the token can pick one of the available tasks such that its pickup and delivery locations are different from those of currently assigned tasks (Line 7). If at least one such task exists, the agent chooses the task τ having the pickup closest to its current location, according to a heuristic h (e.g., the Manhattan distance in case of grid environments), and removes the task from the set $\mathcal{T}$ (Lines 9–11). The agent's path in the token is updated by computing, using the path planning function PP, a minimal cost path that reaches the delivery location of τ, passing through the pickup location, while avoiding any conflict with the paths of other agents stored in the token (Line 12).

If the agent cannot be assigned to any task, then it is allowed to idle in its current location only if it is not occupying the delivery location of any other task in the tasks set (Lines 13–14). Otherwise, the token is updated with a minimal cost collision-free path (calculated using the path planning function $PP2$) that moves it to a non-task endpoint that is not currently scheduled to be occupied by another agent (Line 16). Finally, the token is returned to the system, and each agent moves one step along its planned path in the token (Line 21). When a task is completed, the corresponding path is removed from the token and the agent becomes free (not shown in the pseudo-code).

3. Related work

In this paper, we present a new formulation that extends MAPD to consider the presence of external agents, that is a case of interest for some applications: for this reason, we first survey the literature about MAPF and MAPD variants. Then, we present some MAPF and MAPD algorithms that share some similarities with our approach. Given that our solution is based on the behavioral modeling of other agents, a brief review of the literature related to this topic is also included. Finally, we place our work in the context of similar works from different fields.

3.1. MAPF and MAPD variants

MAPF and MAPD are relatively young problems, but several variants have been proposed in the literature to widen their applicability and to fill gaps between abstract formulations and real-world applications.

Research has mostly focused on the MAPF setting, by considering approaches that include agents with geometric shapes and volumes [13–15], agents that move at different speeds and with different orientations [16–18], and with continuous time [14,19]. For what concerns the MAPD problem, Ma et al. [20] propose TP-SIPPwRT, a TP variant that takes into account kinematic constraints in the planning phase and computes continuous paths for agents with given velocities. TP-SIPPwRT employs a combinatorial search algorithm to plan paths for single agents, called Safe Interval Path Planning with Reservation Table (SIPPwRT), that uses an advanced data structure to efficiently check and update the paths of agents in an online setting. The resulting MAPD algorithm is complete for well-formed instances.

To move towards more realistic scenarios, authors in [21] consider the case of MAPD in non-uniform environments (N-MAPD), i.e., where paths' width is not assured to be regular, implying that some actions may not be allowed in certain locations. They propose a solving algorithm for N-MAPD called Path and Action Planning with Orientation (PAPO) that takes into account the agent's size, direction, orientation, and the cost and timing of rotations, in addition to environmental constraints.

In [22], MAPD is extended to allow agents to be allocated multiple tasks simultaneously according to their payload capacity, as well as to consider pickup and drop-off costs. An algorithm based on a combination of MAPF and Traveling Salesman Problem is proposed to solve this multi-task MAPD version. Similarly, in [23,24], robots can perform more than one task at the same time. Another formulation of MAPD is that of Multi-Goal Multi-Agent Pickup and Delivery (MG-MAPD), in which each task is composed of a sequence of goal locations, requiring the agents to reach all the goal locations in order [25].

Along other directions, authors in [26] modify TP to include the probabilities of appearance for future tasks, while in [27] TP is extended to take into account possible delays of agents completing pickup and delivery tasks.

Due to the online nature of MAPD, specific challenges arise. One is that of *deadlocks* in non-well-formed environments: to solve it, authors in [28] introduce the concept of standby nodes, where agents can stay for a long time without affecting the connectivity of the graph. Authors in [29] extend the Priority Inheritance with Back-Tracking (PIBT) approach [30] to environments containing dead-ends. The agents plan following a priority order and decide their next moves according to their neighbors, by introducing temporary priorities and limiting agents' moves in dead-ends. In [31], constraints are imposed on agents' movements and on the environment to prevent deadlocks by design.

To the best of our knowledge, no MAPD variant has addressed the setting we consider in this paper, in which two independent groups of agents are operating in the same environment and one group models the behavior of the other group to plan more informed paths.

3.2. Learning-based methods for MAPF and MAPD

In our approach, team agents learn how to cope with the presence of external agents, by modeling their behavior and incorporating it into the planning phase. Recently, several learning-based methods for multi-agent path planning have been proposed in the literature [32]. These methods aim to learn a policy for each agent based mainly on local observations to decide the next step to take. Many of these methods model MAPF as a multi-agent reinforcement learning problem: in [33], authors propose PRIMAL, which combines reinforcement learning and imitation learning to learn a fully decentralized policy for agents based on local observations. In [34], a deep Q-network guided by a single-agent heuristic is proposed, which embeds all the potential choices of shortest paths for a single agent and thus guides the network. Agents communicate with neighbors through graph convolution: each agent is represented as a node in a graph, with edges indicating communication links between agents within each other's field of view. The graph convolution operation then updates each agent's state by aggregating information from its neighboring nodes (agents). This method is called DHC, i.e., Distributed, Heuristic, and Communication, which are the three elements it is based on. Another distributed reinforcement learning approach is presented in [35], with the introduction of a PrIoritized COmmunication learning method (PICO), which incorporates the implicit planning priorities into the communication topology. In [36], instead, it is proposed a variant of MAPF called partially observable MAPF (PO-MAPF), in which agents do not communicate and can observe other agents and the environment only within a limited field of view. The authors propose three different approaches for solving PO-MAPF: one is based on heuristic search, one on reinforcement learning, and the other, which proves to be the best one, switches between the two.

A different approach is illustrated in [37], which comprises a convolutional neural network (CNN) and a graph neural network (GNN). The CNN is employed to extract meaningful features from agents' local observations, while the GNN shares this information among agents. The two networks are jointly trained to imitate a centralized planner. A subsequent work [38] takes a step forward: the GNN is extended to incorporate a mechanism that allows message-dependent attention, determining the importance of each feature when agents share information.

While most of the existing approaches focus on MAPF, Damani et al. [39] propose PRIMAL2, extending PRIMAL to lifelong MAPF (LMAPF), in which goal locations are continuously assigned to the

agents. The aim of this framework is for agents to learn fully decentralized policies to reactively follow paths online in a partially observable world. This work relies on the Asynchronous Advantage Actor–Critic (A3C) algorithm, while some strategies are integrated to avoid agents acting selfishly. LMAPF is also tackled in [40], where authors combine a heuristic-based planner with reinforcement learning.

Although our work shares some similarities with these approaches, none of them deals with our variant of the MAPD problem, and some of them make limiting assumptions (e.g., [33] assumes agents take actions one at a time). In a nutshell, these methods aim to learn decentralized policies, while we aim to allow agents to model what surrounds them, and exploit this information while planning in a coordinated way to accomplish MAPD tasks.

3.3. Modeling other agents

Generalizing beyond MAPD, the development of autonomous agents able to effectively interact with others is a topic of much interest nowadays. Agents construct models of others by observing their conduct or exploiting past interactions, and then use these models to make predictions about the future behavior of other agents to make more informed decisions. Underlying assumptions about the modeled agents and the environment may vary, as explained in [41]: setting these choices can help to have a clearer view of the problem. For example, it is useful to know whether the modeled agent makes deterministic or stochastic action choices, or if it has a static or dynamic behavior (i.e., a behavior that changes over time). When modeling a group of agents, they may have common or conflicting goals, and they may base their actions on the decisions of the others or be independent. For what concerns the environment, the most important assumption is its degree of observability: it can be fully observable, meaning that the agent has a complete knowledge of what is happening, or partially observable.

Several techniques have been developed to model different aspects of other agents under different assumptions [41]. Classification methods can be used to label the behavior of an agent into a predefined category, as in [42], in which a hierarchically structured model is used to classify the behavior of the opponent in a real-time strategy game, or as in [43], in which opponent modeling is used to classify the attack and defensive strategies of the opponent team in a football game.

Plan and goal recognition techniques aim at identifying the intended goals of the modeled agents and their possible future steps to reach such goals according to the observed behavior. Many plan recognition techniques employ hierarchical plan libraries [44,45], which decompose a complex action into a hierarchy of simple actions, obtaining a graph-like structure of the possible plans and transforming the problem into a graph covering task. Another approach is proposed in [46] and is based on the fact that, given the set of possible goals, each rational agent follows the optimal path to its objective, and its goal is identified as the one whose optimal path is more similar to the path followed by the agent. When the aim is the reconstruction of the policy of the agents, one simple yet effective approach is "fictitious play" [47], in which an agent is modeled according to the statistical empirical distribution of its past actions. Although we do not plainly use any of the above models, we borrow some general ideas from them, such as relying on the distribution of agents' past actions and aiming at reconstructing possible future plans of the agents.

3.4. Other relevant topics

Although our setting is composed of two different groups of agents in the same shared environment, we do not end up neither in the adversarial planning case [48,49] nor in the case of cooperating heterogeneous teams of agents [50]. Team agents have to accomplish their MAPD tasks without interfering nor communicating with external agents. For this reason, external agents are not affected by the presence of team agents. This is similar to [51–53], which address autonomous aerial vehicles' task allocation and trajectory planning in dynamic environments, focusing on real-time target tracking and collision avoidance. As in our MAPD-EA setting, agents must dynamically reassign tasks and adjust paths to account for moving obstacles, but the purpose is to track the targets and not accomplish a stream of pickup and delivery tasks.

Another setting similar to ours could be found in the field of robot–human interaction. Robots are increasingly employed in densely human-populated environments, so they must be able to perform their tasks and at the same time not interfere with humans' actions. For example, in [54,55], a service robot is required to operate in an environment highly populated by pedestrians: after an observation phase, it uses a spatio-temporal representation to model how the human flow changes over time. In [56], service robots use a sequence-to-sequence LSTM (Long Short-Term Memory) encoder–decoder to predict human trajectories and act accordingly. The main difference between our setting and the human–robot interaction field lies in the fact that we model artificial systems, not natural systems, which usually exhibit strong periodic behaviors. Indeed, many existing modeling approaches [54,55,57] prioritize extracting temporal patterns and periodicities associated with the human behavior, building different models that are used for planning according to the time of day [58,59]. For instance, social robots operating in office environments must distinguish between weekdays and weekends, as well as day and night. However, in application cases relevant to our setting, such as automated warehouses, spatial patterns are more relevant. Besides, these studies typically focus on planning for a single agent, whereas we address the coordination of a team of agents.

Our work is also related to the problem of multi-agent moving obstacles' avoidance, but differs in the important aspect that in the latter it is not always assumed that the obstacles follow a behavior that is worth modeling over an extended period. In [60,61], the focus is only on the identification and avoidance of obstacles, not on the possibility of identifying long-term patterns in obstacles' movements and planning accordingly. Still, in other works [62], the movement of the obstacles is taken into account, but mainly from a kinematic point of view: employing a kinematic model, a probability of occupation can be assigned to different locations of the environment close to the robots.

4. Problem formulation

In this section, we first define our variant of the MAPD problem, where a team of agents must perform pickup and delivery tasks in a dynamic environment populated by other agents. Then, we present a simple variation of the Token Passing algorithm that, when run by team agents, tries to reactively avoid collisions with external agents when they appear.

4.1. MAPD with external agents

The main difference between classical MAPD and our variant is that the environment, from the point of view of the team agents, is not static, but changes over time. This dynamicity arises from the presence of other external agents that move in the same environment as the team agents. The paths and goals of external agents are unknown to team agents, that have to complete MAPD tasks while avoiding interfering with external agents. We call this variant *MAPD with External Agents* (*MAPD-EA*).

More formally, we consider a team of k agents, $\mathcal{A} = \{a_1, \dots, a_k\}$, referred to as *team agents*, which operate in an environment modeled as an undirected connected graph $G = (V, E)$. In our experiments, we consider grids, but our approach applies to generic graphs. Team agents can communicate with each other and have to perform pickup and delivery tasks that appear online and are collected in a set $\mathcal{T}$ while having full knowledge of the structure of the environment G. In a classical MAPD setting, their objective would be to complete all the pickup and delivery tasks in $\mathcal{T}$ without collisions while minimizing

a cost measure, such as service time or makespan. In MAPD-EA, we additionally consider the external agents.

More precisely, in the same environment, there is another set of n *external agents*, $\mathcal{M} = \{m_1, \dots, m_n\}$. These external agents move to perform their own activities without colliding between them. We assume that the team agents and the external agents are neither cooperative nor adversarial and that they are unaware of each other's goals and paths. Additionally, we make a *no-interference* assumption, meaning that the activities and plans of the external agents are fixed and cannot be changed. Therefore, it is responsibility of the team agents only to implement behaviors to avoid collisions. If a team agent a_i, while following its path π_i, detects a potential collision (i.e., a vertex or swapping conflict) with an external agent m_j, a_i must adjust its planned path to avoid the collision, as the external agent m_j will not take any action to prevent the collision.

To allow team agents to detect potential collisions with external agents, we assume that each team agent $a_i \in \mathcal{A}$ can detect external agents within its field of view $FOV(a_i) = \{\ell \in V \mid \exists \pi = (loc(a_i), \dots, \ell) \text{ with } |\pi| \leq 2\}$. This field of view encompasses all locations ℓ that are reachable from the current location of a_i by paths π of length 2 or less (excluding wait actions). This choice is quite conservative (making our problem more challenging), considering that in other works that propose learning-based methods for MAPF, the field of view has radius equal to 5 [36] or 11 [35,39] cells. Team agents are assumed to know the locations and the next actions of external agents within their field of view. For instance, in a warehouse setting, this means that team robots can detect the current heading of external robots in their field of view and that external robots are equipped with turn signals, a feature common in many warehouse robots. When a team agent a_i observes the next actions of external agents within its field of view, it will continue along its path π_i if there is no collision. However, if a collision is going to happen based on the current path π_i and the observed next actions of the external agents in $FOV(a_i)$, the team agent a_i must alter its planned action to avoid the collision (refer to Section 4.2 for details).

To summarize, we assume that a team agent a_i can communicate and coordinate with other team agents, but at each time step it can only observe the location and next action of those external agents within its field of view $FOV(a_i)$. The activities and plans of external agents are unknown to team agents. The objective of the team agents is to complete all their pickup and delivery tasks as fast as possible (i.e., minimizing the makespan) without interfering or colliding with external agents (and with other team agents).

A number of non-task endpoints equal to the number k of team agents is present in G, so that well-formedness is guaranteed at least for the team agents. We assume that both team and external agents are synchronized with the same global clock and that the discrete time steps have the same (unitary) duration.

Finally, note that in the case of no external agents (i.e., $n = 0$), our problem reduces to regular MAPD as a special case, implying that MAPD-EA is at least NP-hard.

4.2. TP with collision avoidance and replanning

Team agents do not know external agents' plans in advance, implying that collisions between an external and a team agent may happen. One straightforward way to deal with this issue is to impose team agents to change their paths whenever they detect a potential collision in their field of view. To this end, we propose a variant of the TP algorithm to be run by team agents in which, before moving, a team agent checks if its planned action would result in a collision and, if so, it modifies its path. This approach is purely reactive: we will present a more proactive approach in the next section. Note that we are agnostic of the planning algorithm (if any) used by external agents, i.e., they can use any planning algorithm.

Algorithm 2 TP with collision avoidance and replanning + Model

1: */* system executes now */*;
2: Initialize *token* with the (trivial) path $(loc(a_i))$ for each team agent a_i ($loc(a_i)$ is the current location of a_i);
3: **while** true **do**
4: Proceed like in Algorithm 1 (lines 4 - 19);
5: **for all** team agents $a_i \in \mathcal{A}$ **do**
6: $C \leftarrow CheckCollisions(loc(a_i))$;
7: **if** $C = \emptyset$ **then**
8: a_i moves along path in *token* for one time step;
9: **else**
10: a_i makes the $BestLegalAction(a_i, loc(a_i), C, token)$;
11: $\pi_i \leftarrow PP(a_i, \tau, token, \hat{G})$ */* replan */*
12: **end if**
13: **end for**
14: */* system advances to the next time step */*;
15: **end while**

Algorithm 2 (excluding blue parts) presents the *TP with collision avoidance and replanning* (*TP-CA*) algorithm, which follows the same structure as the TP algorithm introduced in Section 2.3 (see Algorithm 1), but with a modified action execution process. In TP-CA, before making a move, a team agent checks whether the move would cause a collision with nearby external agents (Line 6). A collision is defined as a situation when the next move of an external agent within $FOV(a_i)$ results in both agents occupying the same location or swapping locations.

If a future collision is detected, the team agent a_i selects the best possible legal action executable from $loc(a_i)$ that brings it closer to its pickup or, if it has already performed the pickup, to its delivery. Legal actions are those that do not result in a collision with any external agent, another team agent, or an obstacle (Line 10). After executing the action and avoiding the collision, a_i updates the token with a new path to its goal starting from its new location (Line 11). Note that reversing the last action (the one taken to avoid the collision) and resuming the original path is in general infeasible, as the delayed path might conflict with the paths of other team agents. Fig. 4 (left) illustrates a situation where the team agent a_1 cannot proceed with its next planned move because it would collide with external agent m_1. The collision is avoided by a_1 moving left, its only legal action.

We note that there are situations where no legal action can prevent a collision with an external agent. These situations are referred to as *deadlocks*, and their occurrence depends on the environment's configuration and the number of external agents. Fig. 4 (right) shows a deadlock scenario where a_1 has no legal action to avoid colliding with m_1. Staying still would result in a vertex conflict with m_2, while moving down would cause a swapping conflict with m_2. To prevent deadlocks, also external agents would need to implement reactive policies, which contradicts the no-interference assumption. In another work [31], we propose a method that prevents deadlocks by design for MAPD-EA by imposing constraints on the environment and the movements of team and external agents. However, in order to consider any type of environment and to leave freedom of movement to the agents, we do not apply it in this paper, allowing deadlocks to happen (see Section 8).

5. Planning with a model

The main issue with following a purely reactive approach like TP-CA for solving a MAPD-EA problem is that, when a future collision is detected, we incur the cost of replanning the path of one or more team agents. A more proactive approach is proposed in this paper and illustrated in this section, in which particular care is devoted to planning paths that keep the number of conflicts at bay. To this purpose, a model of external agents is constructed to provide team agents with

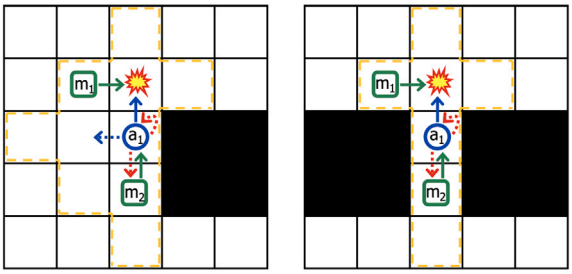


Fig. 4. Examples of collision avoidance (left), and deadlock (right). Team agent a_1 is the blue circle, external agents m_1 and m_2 are green squares. Solid arrows represent the next planned action of each agent, while dashed arrows are legal (blue) and illegal (red) possible actions for team agent a_1. The orange dashed boundary is $FOV(a_1)$.

the information they need to plan a convenient sequence of actions for fulfilling their tasks while avoiding collisions with external agents. This approach is called TP with Collision Avoidance and Replanning + Model (TP-CA-M) and is presented in Algorithm 2 including the blue parts.

Our proposed approach uses TP-CA with new path planning functions (Lines 12 and 16 of Algorithm 1 and Line 11 of Algorithm 2) that exploits models that provide an estimate of the probability of collisions between team and external agents at each time step.

Path planning algorithms called in Lines 12 and 16 of Algorithm 1 and Line 11 of Algorithm 2 are classical single-agent path planners that return minimum cost paths that do not collide with paths of other team agents in the token. Team agents should search for paths that, ideally, are the shortest, do not collide with those of other team agents, and minimize the occurrence of collisions with external agents. To combine these two objectives we introduce a weighted directed graph $\hat{G} = (V, \hat{E})$, derived from the original graph G, where $\hat{E}$ contains the same edges of E but with specific weights. This formalization allows us to optimize for a single objective. The weight w_{ij} of an edge $(v_i, v_j) \in \hat{E}$ is defined as the convex combination of two cost functions, governed by a parameter α. Specifically:

$$w_{ij}(t) = (1-\alpha) \cdot \frac{1}{diam(G)} + \alpha \cdot p_c(v_i, v_j, t). \tag{1}$$

The first term accounts for the time needed to complete tasks, and its contribution is equal to the distance (cost) between vertices v_i and v_j (which is 1), normalized by the factor $diam(G)$, which is the graph diameter. This normalization ensures that the magnitudes of the two cost functions are comparable. The contribution given by the second cost function is in fact limited between 0 and 1, as it represents the probability p_c of incurring a conflict while traversing the edge (v_i, v_j). In principle, this probability is not constant over time, but evolves as external agents move within the environment, so we make explicit the dependence of p_c on the time step t at which the edge (v_i, v_j) is planned to be traversed.

From an algorithmic perspective, the main difference between TP-CA and TP-CA-M is the use of the weighted graph $\hat{G}$ in the path planner functions PP and $PP2$. It should be noted that the introduction of these weights in $\hat{G}$ makes it hard to define an admissible (non-trivial) heuristic function for a path planning algorithm such as A*. Therefore, we use the Dijkstra algorithm as path planner.

In the following sections, two approaches for modeling the probability of conflicts p_c are presented: the first one relies on an occupancy estimate to assess at which vertices external agents are likely to be, while the second one adopts Markov chains for predicting future movements of external agents.

Runtime conflicts due to model inaccuracies are still possible, so a reactive conflict resolution mechanism cannot be avoided. For this reason, as shown in Algorithm 2, our solving algorithm has the same structure as TP-CA, with an improved path-planning procedure that takes advantage of the model of external agents.

6. Occupancy model

The first approach we present for estimating p_c has been introduced in our previous work [8]. With this approach, the behavior of external agents is represented by means of a probability of occupancy for each vertex of the graph. The general idea is to model the occupancy of a vertex $v_j \in V$ as the realization of a Bernoulli process with parameter p_j, assuming independence between processes of different vertices. In this setting, the probability of encountering a vertex conflict while traversing edge (v_i, v_j) coincides with the parameter of the underlying stochastic process at the destination vertex v_j. Conversely, the probability of a swapping conflict cannot be immediately derived from this formalization; therefore, the choice here is to disregard this type of conflict, which is not modeled and is accounted for only reactively at runtime. Thus, our occupancy model delivers the information about the probability p_c of incurring conflicts while traversing an edge, capturing just the probability of occurrence of vertex conflicts. Moreover, in this model, the probability $p_c(v_i, v_j, t)$ does not depend on the source vertex v_i nor on the time step t of the edge traversal, as the stochastic process is assumed to be stationary, namely $p_c(v_i, v_j, t) = p_c(v_j)$.

The occupancy probability for each vertex can be estimated by collecting observations on the locations of external agents in the environment. Specifically, the occupancy probability $p_c(v_j)$ of vertex $v_j \in V$ is computed as the ratio of time steps in which v_j is observed to be occupied by some external agent and the time steps composing the observation period T. Notice that we are implicitly assuming complete observability of the movements of external agents during the observation period. In practice, this can be achieved via monitoring cameras covering the whole environment or by having team agents patrolling strategic locations. We acknowledge that this requirement may not be practicable in some applications. Nevertheless, it sets the ideal conditions for estimating the occupancy model parameters and allows to assess the full potential of our approach. For this reason, the experiments presented in Section 8 are conducted in this ideal scenario. In Section 9, we relax this requirement to reflect more realistic settings in which the model is built incrementally.

The assumptions underlying the occupancy model are likely too simplistic to capture complex behavioral patterns of external agents. In the following section, we explore more elaborate models for the behavior of external agents.

7. Markovian models

The second type of models we present to represent p_c refines the previous one by shifting the focus to estimating the movements of the individual external agents, rather than representing the behavior of the entire group as a whole. We develop a probabilistic model that captures the temporal evolution of the external agents' locations, moving beyond the occupancy estimate, which is stationary by nature. To this purpose, we describe the movement of the external agents within the environment by a nth order Markovian stochastic process [63]. The location occupied by an external agent at a time step in the future is thus assumed to be dependent only on the n most recently occupied locations.

This way of formalizing an external agent's behavior effectively encodes the temporal dynamics of its movement into a set of transition probabilities that can be estimated from data. Moreover, team agents can reason about the possible future locations of an external agent by propagating the belief on its current location forward in time.

In the rest of this section, we formalize how we maintain a representation of the individual external agents' locations under uncertainty, and present a way of aggregating these representations into an estimate of the group of external agents' probability of occupancy that can inform the planning of the team agents. Then, we propose an update rule for predicting the current locations of the external agents that incorporate the information from the fields of view of the team agents.

To this purpose, it is assumed that external agents appearing within the fields of view can be individually identified at all times (e.g., by tags), as this avoids ambiguity when updating their representations. Finally, we conclude this section by discussing a solution for estimating the transition probabilities from data and by assessing the computational complexity of our approach.

7.1. Belief states

In general, the exact location of each external agent is known only when it is within the field of view of a team agent, making the movement of the external agents a partially observable process from the perspective of team agents. For this reason, the information that team agents possess on the history of external agents is affected by uncertainty. Formally, the history of an agent is the sequence of its n most recently occupied vertices of the environment, where n represents the order of the Markov process. The uncertainty on the history of an external agent $m \in \mathcal{M}$ at time step t is explicitly quantified using a *belief state* $b_m^{(t)}$: a probability distribution over sequences of vertices of length n, expressing the likelihood of each sequence being the history of agent m at time step t. The *belief space* $\mathcal{B}^m$ is the space of all possible belief states for the external agent m:

$$\mathcal{B}^m = \left\{ b_m : V^n \to [0,1] \;\middle|\; \sum_{h \in V^n} b_m(h) = 1 \right\}.$$

Starting from the belief state $b_m^{(t)}$ at the current time step t, we can predict how this distribution will evolve after s time steps using the Chapman–Kolmogorov equation:

$$b_m^{(t+s)} = \left(P_m{}^{\top}\right)^s \; b_m^{(t)}, \tag{2}$$

where $P_m = [p_{ij}]$ is the transition matrix of the Markov process, whose element p_{ij} denotes the probability that the history of agent m will evolve from h_i to h_j in one step, and $P_m{}^{\top}$ is the transpose of P_m. Notice that p_{ij} coincides with the probability $p(v|h_i)$ of transitioning to vertex v given a history h_i only if h_i and h_j are compatible, meaning that $h_i = (v_1, v_2, \dots, v_n)$ and $h_j = (v_2, \dots, v_n, v)$. In all other cases, p_{ij} is zero.

For the sake of maintaining a simple notation, in the remaining part of this section we will provide all the definitions and formulas for a Markov process of the first order ($n = 1$), but the formulation can be extended to the nth order.

7.2. Estimating the collision probability p_c

Recalling Section 5, the model of external agents is needed to estimate $p_c(v_i, v_j, t)$ used in Eq. (1). In this section, we show how this is done in the case of Markovian models.

Under some independence assumptions, we can derive an expression for p_c that depends on the belief states b_m of the individual external agents. Specifically, by assuming independence between the occurrence of vertex and swapping conflicts, we can express p_c as:

$$p_c(v_i, v_j, s) = 1 - \Big[\big(1 - p_v(v_i, v_j, s)\big) \big(1 - p_e(v_i, v_j, s)\big) \Big],$$

where p_v denotes the probability of incurring in a *vertex conflict*, and p_e denotes the probability of incurring in a *swapping conflict* along an edge (recall that this type of collision is not accounted for in the occupancy model of Section 6).

When a team agent traverses the edge (v_i, v_j) at time step $t+s$, there is not any vertex conflict only if no external agent is in v_j at time step $t+s$, therefore – assuming independent external agents' positions – we can write:

$$p_v(v_i, v_j, s) = 1 - \prod_{m \in \mathcal{M}} (1 - b_m^{(t+s)}(v_j)).$$

Similarly, traversing edge (v_i, v_j) does not result in a swapping conflict only if none of the external agents moves from v_j to v_i at time step $t+s$. Therefore, we write:

$$p_e(v_i, v_j, s) = 1 - \prod_{m \in \mathcal{M}} (1 - b_m^{(t+s-1)}(v_j) \cdot p_{ji}).$$

7.3. Updating belief states

In this section, we discuss how the current belief states for external agents are actually obtained.

Given some initialization of the belief state $b_m^{(0)}$, we could simply compute the predicted distribution after t time steps using (2) and adopt it as the belief state for the current time step $b_m^{(t)}$. This, however, would completely disregard the additional information collected at each time step from the fields of view of the team agents. For this reason, we choose to update the belief state at each time step in an iterative fashion, incorporating this information.

After an external agent m has moved one step, it is either visible or not to the team agents. In the former case, its location $loc(m)$ is within some team agent's field of view, so there is no uncertainty regarding its position. Therefore, its updated belief state will be equal to:

$$b_m^{(t+1)}(v) = \begin{cases} 1, & v = loc(m) \\ 0, & v \neq loc(m). \end{cases}$$

In the latter case, the external agent's location is still uncertain, nevertheless, we can safely assign zero probability to the set F of all vertices belonging to at least one team agent's field of view. For the updated belief state, we can compute the predicted distribution one step in the future, ruling out the vertices in F. Formally:

$$b_m^{(t+1)} = \tilde{P}_m^{\top} b_m^{(t)},$$

where $\tilde{P}_m = \left[\tilde{p}_{ij}\right]$ is a stochastic matrix derived from P_m such that:

$$\tilde{p}_{ij} = \begin{cases} 0, & v_j \in F \\ p_{ij} / \sum_{v_k \in V \setminus F} p_{ik}, & v_j \notin F. \end{cases}$$

7.4. Estimating transition probabilities p_{ij}

The last aspect to be discussed is how the transition probabilities p_{ij} of the Markov process can be obtained. The nature of the problem mandates that a non-zero transition probability be assigned only to those pairs of vertices that are adjacent in the graph G. However, the values of the transition matrix ultimately depends on the behavior of the external agents and on their activities within the environment. In these settings, an estimate for the transition probabilities can be built from past observations of the external agent's behavior, under the assumption that these data are representative of the agent's future behavior. In principle, our approach allows fitting a set of transition probabilities P_m for each one of the external agents. However, for simplicity, in our experiments, we estimate a single transition matrix P_m for all external agents $m \in \mathcal{M}$ (that is to say that they are assumed to exhibit similar behaviors).

We learn the transition probabilities using a maximum likelihood estimator. Let X_t be the state variable of the Markov chain representing the location of an external agent, and let $\mathcal{S}$ be the set of possible states that X_t can take on. Assuming we have collected a sequence $X = (X_0, \dots, X_h)$ of observations, let us define:

$$N_h^{ij} = \sum_{t=1}^{h} \mathbb{1}_{\{X_{t-1}=i, X_t=j\}}$$

as the number of one-step transitions from v_i to v_j which were observed until time step h. Let $\Theta = \left(p_{ij}, (v_i, v_j) \in \mathcal{S}^2, i \neq j\right)$ be the unknown parameter of the transition probabilities. It can be shown that the maximum likelihood estimator is given by $\hat{\Theta} = (\hat{p}_{ij})$ with:

$$\hat{p}_{ij} = \frac{N_h^{ij}}{\sum_{k \in \mathcal{S}} N_h^{ik}} \quad (i,j) \in \mathcal{S}^2. \tag{3}$$

7.5. Algorithm analysis

In this section, we provide a theoretical analysis of our algorithms.

7.5.1. Computational complexity

Here, we evaluate the computational complexity of our TP-CA-M algorithm (Section 5). TP-CA-M, as TP, operates on a graph with nodes $\mathcal{V}$ and edges $\mathcal{E}$. Each node is represented as a pair (v, t), where $v \in V$ denotes a spatial location and t a discrete time step. Directed edges in $\mathcal{E}$ are $((v, t), (v', t+1))$, with $v = v'$ or $(v, v') \in E$. Note that $|\mathcal{V}| = |V| \cdot T$ and $|\mathcal{E}| = |E| \cdot T$.

For single-agent path finding on such a graph over T time steps, the Dijkstra algorithm has a complexity of $O((|V| \cdot T)\log(|V| \cdot T) + (|E| \cdot T))$. In TP, team agents sequentially compute their paths using the Dijkstra algorithm, with a worst-case complexity for the token-passing phase of $O(k \cdot |\mathcal{T}| \cdot ((|V| \cdot T)\log(|V| \cdot T) + (|E| \cdot T)))$, where k is the number of team agents and $|\mathcal{T}|$ the number of tasks.

In TP-CA-M, however, each expansion of the Dijkstra algorithm requires consulting the model to adjust edge weights. If the consultation has a cost of $O(C)$, the complexity of the Dijkstra algorithm increases to $O((|V| \cdot T)(\log(|V| \cdot T) + C) + (|E| \cdot T))$. Consequently, the computational complexity of TP-CA-M becomes $O(k \cdot |\mathcal{T}| \cdot ((|V| \cdot T)(\log(|V| \cdot T) + C) + (|E| \cdot T)))$, assuming a single path computation per team agent. However, if the Dijkstra algorithm is run multiple times (e.g., for replanning), the complexity scales accordingly.

7.5.2. Model impact

The cost $O(C)$ depends significantly on the model used (occupancy or Markovian), which affects performance in three ways: model construction, model consultation, and model updating.

Model construction has a minimal impact on overall performance. The model is constructed at the end of the observation phase with a computational cost that is linear in the number of collected observations. For the occupancy model, observations are grouped by location, and an estimate for the probability of occupancy is obtained as the ratio between the number of times an external agent has been observed in a given location to the total number of observations for that location. In the Markovian model of order n, an estimate of the transition matrix is built from $(n+1)$-tuples of consecutive observations in a similar fashion. Transition probabilities are estimated by counting how many times external agents have followed a path of length $n+1$, divided by the total number of times they have followed the first n steps of that path. If no observation phase is assumed, all approaches consider a uniform distribution as the initial estimate of the probabilities, further reducing the computational effort.

In the previous section, we have indicated the computational cost of consulting and updating the model as a generic $O(C)$. We now detail this cost based on the specific model employed.

The probability occupancy grid can be consulted in constant time to obtain the cost associated with a specific location of the environment. If an observation phase is assumed, the model is constructed beforehand and remains static thereafter, eliminating any update costs. Conversely, for an occupancy model built incrementally, updates occur whenever an external agent enters the field of view of a team agent. The update process follows the same steps used for construction: the number of occurrences of external agents seen at each location is updated, and the ratio for the probability of occupancy is recomputed accordingly. The frequency of updates depends on several factors, including the number of team and external agents, as well as the size and shape of the environment. A higher number of agents increases the frequency of updates, whereas a larger open environment reduces this frequency.

In the Markovian model, the computational effort for model consultation is higher. The calculation of $p_c(v_i, v_j, t)$ now depends on t and is determined by the belief states $\{b_m^{(t)}\}$ of all external agents at the time step t. Computing these belief states, in principle, requires multiplying the tth power of the transition matrix $(P_m)^t$ and the corresponding belief state at the current time step. As a result, the overall cost of planning a path can be prohibitive, as a large number of collision probabilities must be consulted during the expansion of the search tree. In practice, however, the complexity can be greatly reduced by caching the computed belief states for increasing values of t. Specifically, computing the belief states of a given external agent m over s consecutive time steps only requires s iterative multiplications between the transition matrix P_m and the initial belief state $b_m^{(0)}$. In this way, the total cost of the model consultation operations required for planning a path scales linearly with the maximum time step s explored in the search tree and the number n of external agents in $\mathcal{M}$. In addition to model consultation for path planning, the external agents' belief states must be updated at each time step, based on the information coming from the team agents' fields of view. The cost of this update is linear in the number of external agents. If an incrementally built model is considered, the transition matrix is also updated in case of agent encounters. The same considerations on complexity and update frequency apply as in the occupancy model.

A quantitative assessment of the computational effort required by the proposed approaches with varying numbers of tasks and agents is reported in Section 10.2.

8. Experiments

In this section, we describe the experimental setting employed to evaluate the proposed method and the obtained results.

8.1. Experimental setting

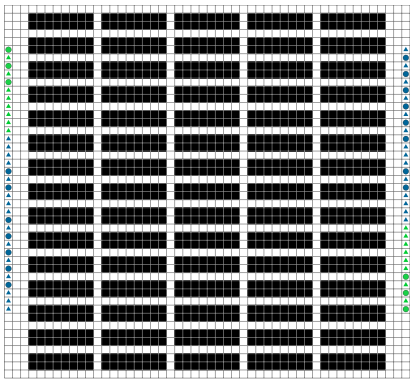
Our approach is tested on grid environments with layout resembling that of warehouses, shown in Fig. 5. Obstacles are depicted by black cells and ideally represent shelves. In this sense, eligible pickup endpoints are all those cells that are immediately North or South of some shelf, while delivery endpoints are on the East and West boundaries of the grid. The first environment (Fig. 5(a)) is a 50×46 grid world featuring one-cell-wide horizontal and vertical aisles, where team and external agents can move freely. The second environment (Fig. 5(b)) is 38×37 and features one-cell-wide horizontal aisles within which the movement of external agents is limited. In particular, external agents shall comply with one-way constraints laid out so that adjacent aisles have opposite directions of travel, while team agents are free to move in every direction. In both environments, the narrow aisles cannot be traversed in opposite directions at the same time: while we assume that inter-team conflicts are prevented through coordination, conflicts between team and external agents can occur and could result in deadlocks. Note that both environments are highly constrained to force frequent and challenging interactions between team and external agents. Moreover, it is important to observe that the individual MAPD instances that result from these environments are not well-formed. This deliberate choice was made to investigate the impact of the models of external agents on the reduction of the number of deadlocks.

In our experimental settings, external agents are engaged with pickup and delivery tasks, and use the TP algorithm to complete them. This is done for simplicity, since external agents could be carrying out activities of any nature, as long as they exhibit a stationary behavior within the environment.

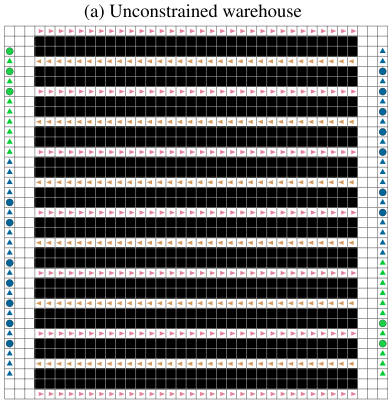
Team agents, in turn, are engaged in a MAPD instance of their own that they solve via the algorithm TP-CA-M presented in Section 5. The models used to capture external agents' dynamics are:

- a probabilistic occupancy model called TP-CA-OM (see Section 6);
- a first-order Markov model ($n = 1$) called TP-CA-MC(1) (see Section 7);
- and a second-order Markov model ($n = 2$) called TP-CA-MC(2) (see Section 7).

For the sake of comparison, we use TP-CA as a baseline that solves conflicts between team and external agents only reactively. To the best of our knowledge, there are no other methods for MAPD-EA against



(a) Unconstrained warehouse



(b) One-way warehouse

Fig. 5. Warehouse environments. Circles represent the initial locations of the agents and constitute the only non-task endpoints, while triangles denote the locations of delivery endpoints. Green and blue colors are used for the team and external agents, respectively. In Figure (b), yellow and pink triangles along the aisles denote the allowed directions for external agents.

Table 1
Experimental parameters.

	Unconstrained	One-way
Size	50 × 46	38 × 37
# instances	100	100
# team agents	6	6
# team tasks	25	25
# external agents	15	15
# external tasks	300	300
Observation window length	300	300

which we can compare. Note that TP-CA represents a traditional purely reactive approach, a widely adopted strategy in multi-robot systems (see Section 3) and is here used as a baseline.

In our problem instances, both team and external agents are initially idling in parking spots (non-task endpoints) of the environment where they cannot interfere with the movement of other agents. The tasks for the external agents are set to appear between time step 0 and time step 750, whereas the tasks for the team agents start appearing only after time step 600 and cease at time step 750 as well. During the initial 300-step window, team agents have complete knowledge about the locations of the external agents and can fit the occupancy and Markov models on these observations (namely, the observation window is $T = 300$ time step long).

In the settings that we discuss here, team agents are thus assumed to have a limited time window of complete observability over the movements of external agents at the beginning of their MAPD instance. A different setting is discussed in Section 9, in which team agents do not possess such information but, rather, can only rely on samples of external agents' movements collected through their field of view as they carry out their tasks.

The characteristics of the problem instances we have set up are summarized in Table 1. Results are averaged over 100 instances, where each instance differs from the others for the locations of the tasks and tasks' appearance times. In the plots, we consider only those instances that do not end in any deadlock for all models and all α values (see Section 5). Experiments are performed on an Intel(R) Xeon(R) E5506 processor with 32 GB of RAM. The tables corresponding to the plots are reported in the Appendix.

8.2. Experimental results

The metrics chosen to assess the effectiveness of our approach are (i) the frequency with which team agents must replan their paths to avoid potential conflicts, (ii) the makespan, and (iii) the number of runs ending in deadlock.

Unconstrained warehouse environment

Fig. 6 refers to the results obtained in the unconstrained warehouse environment with an observation window of 300 time steps. As shown in Fig. 6(a), incorporating the model during the team agents' planning phase reduces the number of potential conflicts compared to TP-CA across all tested α values, particularly when Markovian models are used (for example, for $\alpha = 0.2$, on average, TP-CA-MC(1) has 32 conflicts, TP-CA-MC(2) has 24 conflicts, and TP-CA has 107 conflicts). Regarding makespan, Fig. 6(b) shows that modeling the behavior of external agents does not imply a big increase in makespan of team agents when α is small (for example, for $\alpha = 0.1$, on average, TP-CA-MC(1) needs 335 time steps to complete all tasks, TP-CA-MC(2) needs 350 time steps, and TP-CA needs 338 time steps). This means that our solution is able to find paths that avoid the external agents without forcing long detours. With bigger values of α, there is an increase in the number of time steps required to complete all tasks compared to TP-CA, which grows with larger α values, as higher α emphasizes avoiding external agents over minimizing path length. Fig. 6(c) reports the number of deadlocks, i.e., the number of runs that end in a stalemate situation. For $\alpha = 0.2$ with TP-CA-M(2) there is a decrease (22 deadlocks for TP-CA-M(2) vs. 32 deadlocks for TP-CA), while in all the other cases the number of deadlocks remains stable or higher compared to TP-CA. Deadlocks are not predictable and stronger constraints are necessary to guarantee that they do not form (see Section 4.2 and [31]).

Fig. 7 reports the time in seconds spent on planning and replanning paths of team agents. On the one hand, the consultation of the models in the planning phase of team agents makes our approach slower, especially as α grows and longer paths have to be computed. However, thanks to the strong decrease in the number of replans, we are able to balance such additional time and even be faster in some cases, as it is possible to see for $\alpha = 0.1$ (on average, TP-CA takes 807 s, TP-CA-OM takes 803 s, TP-CA-MC(1) takes 793 s, and TP-CA-MC(2) takes 1061 s). In general, TP-CA-MC(2) is the most computationally demanding approach (see Section 7.5).

The results obtained in the unconstrained warehouse environment demonstrate that incorporating a model of external agents offers advantages in terms of reducing potential conflicts, makespan, and deadlocks, particularly for small values of α and when using Markovian models with order $n = 1$. However, due to the online nature of MAPD and the unconstrained nature of the environment (i.e., external agents can move freely within it), constructing a model that accurately captures the dynamics of these agents proves to be challenging. Although delivery locations are confined to specific areas, pickup locations can be placed almost anywhere, and the tasks performed by external agents vary significantly. This variability makes it difficult to predict their behavior with precision, as it continuously changes in response to their tasks.

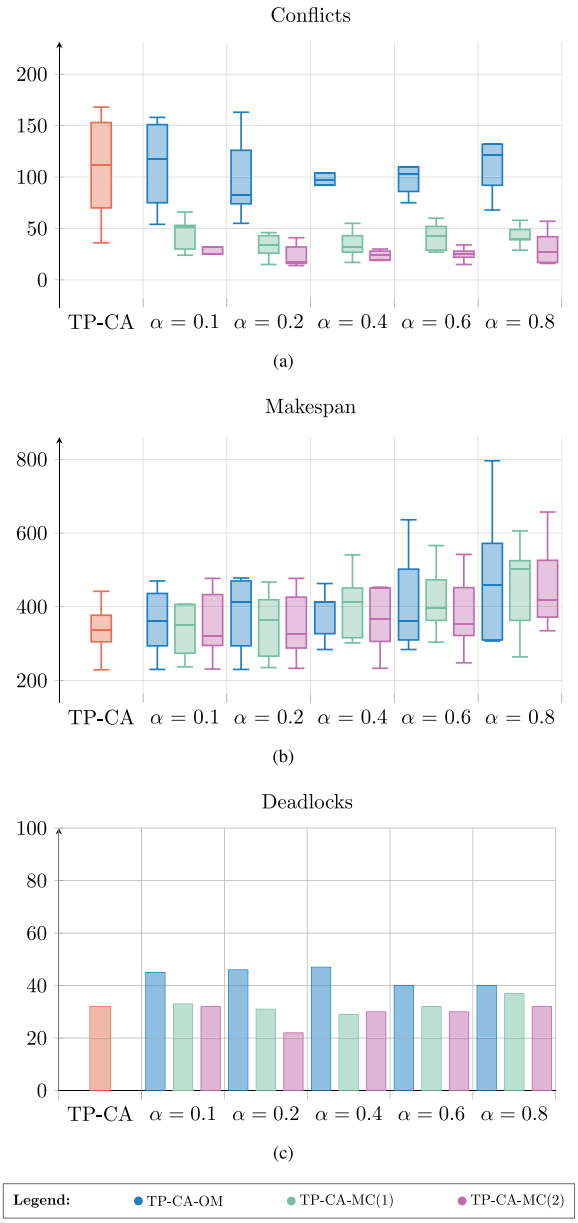


Fig. 6. Number of conflicts (Table A.1), makespan (Table A.2), and number of deadlocks (Table A.3) in the unconstrained warehouse with an observation window of 300 time steps.

One-way warehouse environment

Fig. 8 presents the results for the warehouse environment with one-way constraints and an observation window of 300 time steps. In Fig. 8(a), it can be observed that the number of potential conflicts is consistently lower compared to TP-CA across all models and tested α values. Similar to the unconstrained warehouse, the best results are achieved with Markovian models. Regarding makespan, Fig. 8(b) shows that, for $\alpha = 0.1$, $\alpha = 0.2$, and $\alpha = 0.4$, the makespan is slightly shorter than that of TP-CA. The best results are obtained for $\alpha = 0.1$, where the makespan is on average 508 time steps for TP-CA-OM, 494 time steps for TP-CA-MC(1), and 491 time steps for TP-CA-MC(2) that are compared against 527 time steps for the purely reactive approach TP-CA. As for the unconstrained environment, the makespan increases as α grows, but in a more controlled way. However, a reduction in both task completion time and the number of potential conflicts is observed, indicating that our strategy is quite effective when external agents exhibit structured behavior. This conclusion is further supported

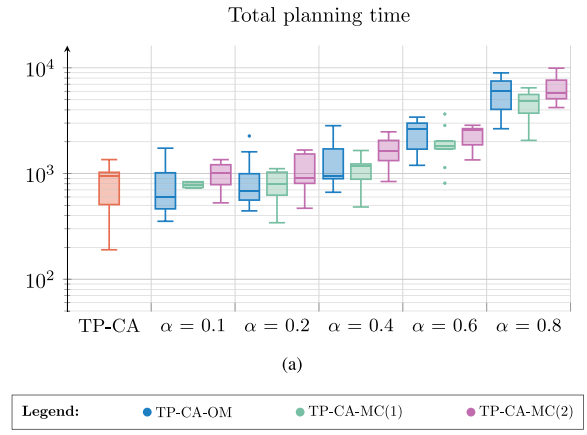


Fig. 7. Total time in seconds spent on planning and replanning paths of team agents in the unconstrained warehouse with an observation window of 300 time steps (Table A.4). The y-axis follows a log-scale.

by Fig. 8(c), which illustrates the number of deadlocks. For all tested values of α, the use of TP-CA-MC(1) and TP-CA-MC(2) always results in fewer deadlocks compared to TP-CA, whereas TP-CA-OM appears to offer limited benefit in this regard.

Fig. 9 shows the total planning and replanning time for team agents in this setting. The results exhibit a similar pattern to that observed in the unconstrained warehouse environment. The strong decrease in the number of conflicts is able to balance (for small values of α) the additional computational burden introduced by the model consultation for both TP-CA-OM and TP-CA-MC(1), which, for $\alpha = 0.2$, have on average a planning time of 609 s and 434 s, respectively, against the 627 s of TP-CA. The TP-CA-MC(2) approach is still slower, with a planning time of 818 s.

9. Incremental model construction

In many real-world scenarios, it may not be reasonable to assume to have access to a long sequence of observations of movements of external agents before team agents can start planning. For this reason, we investigate the performance of our proposed approach when no initial knowledge is provided. In this setting, team agents shall build their model of the external agents incrementally from scratch, based only on the direct observations made when sporadic encounters happen, resulting in potentially sparse and incomplete data (at least at the beginning).

In this paper, we assume that the observations of the external agents collected by the team agents are correct. If sensors are noisy or faulty, team agents can enhance the reliability of the data used to incrementally build models of external agents by aggregating their observations to compensate for missing or inaccurate data. Additionally, techniques like Kalman filters or particle filters can help manage high variance in observations by smoothing estimates and reducing the impact of uncertainty, increasing the robustness of the models of external agents. In general, building such models using uncertain data can require more observations and take longer.

9.1. Experimental setting

The objective is to evaluate the proposed models' effectiveness in capturing the behavior of external agents when prior observations of their actions are unavailable, relying instead on real-time, direct interactions. In practice, each time an external agent enters the field of view of a team agent, the model is updated accordingly.

The experimental setup remains consistent with that of Section 8.1, enabling a direct comparison of model performance with and without

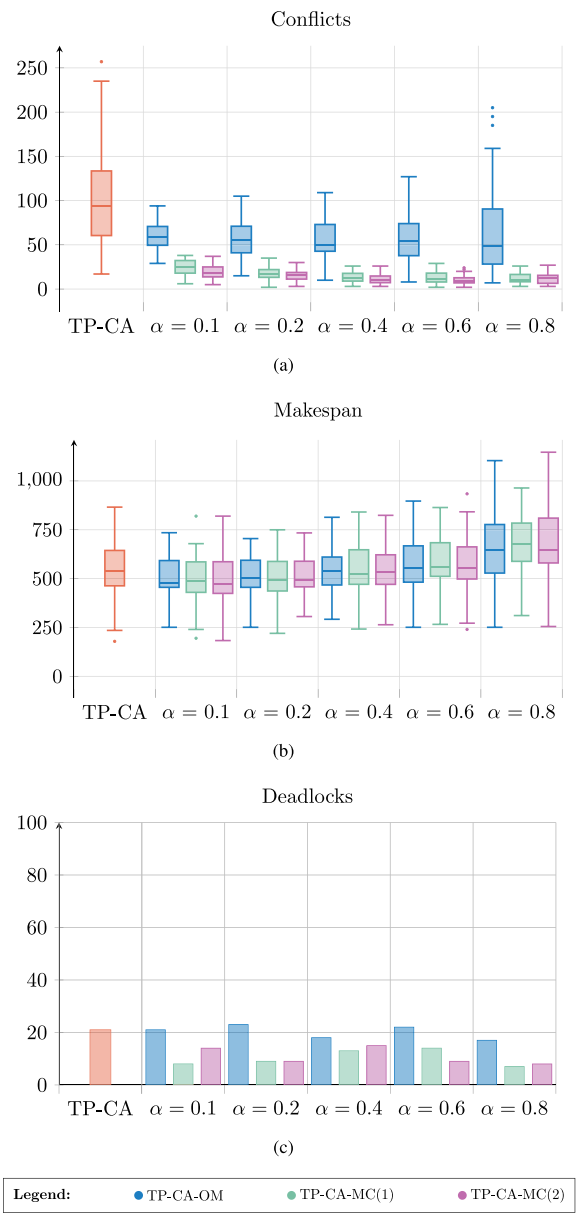


Fig. 8. Number of conflicts (Table A.5), makespan (Table A.6), and number of deadlocks (Table A.7) in the warehouse with one-way constraints with an observation window of 300 time steps.

the observation window. For completeness, in addition to TP-CA-OM, TP-CA-MC(1), and TP-CA-MC(2), the reactive approach TP-CA is also included as a baseline in our plots, although it is not influenced by the absence of the observation period.

Regarding the parameter α, we set it to 0.2. Since there is no observation period, team agents must encounter external agents to gather information about their behavior. Setting $\alpha = 0.2$ reduces the emphasis on avoiding external agents, as focusing too much on avoidance would be less beneficial under the new conditions. The Appendix reports the tables with the numerical values corresponding to the plots.

9.2. Experimental results

The metrics chosen to assess the effectiveness of our approach are (i) the frequency with which team agents must replan their paths to avoid potential conflicts, (ii) the makespan, (iii) the number of runs ending in a deadlock, and (iv) the service time.

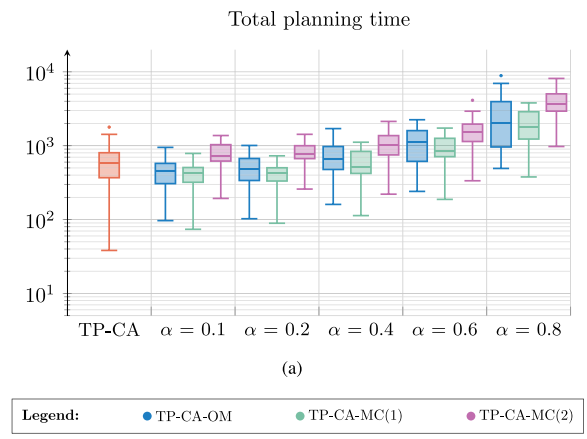


Fig. 9. Total time in seconds spent for planning and replanning paths of team agents in the one-way warehouse with an observation window of 300 time steps (Table A.8). The y-axis follows a log-scale.

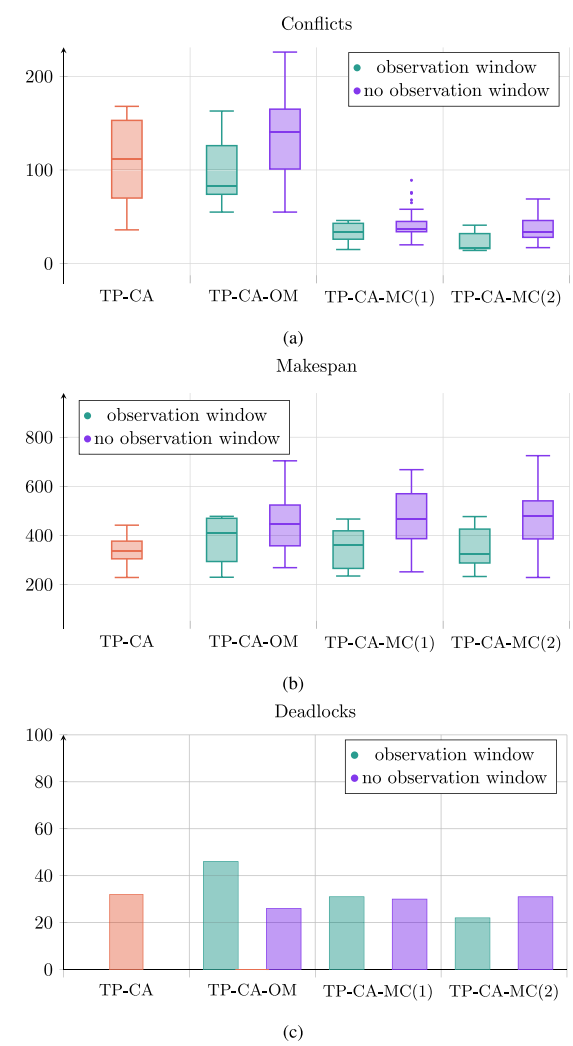


Fig. 10. Performance comparison of number of conflicts (Table A.9), makespan (Table A.10), and number of deadlocks (Table A.11) with and without the observation window for the unconstrained warehouse ($\alpha = 0.2$).

Unconstrained warehouse environment

Fig. 10 reports the comparison between TP-CA-OM, TP-CA-MC(1), and TP-CA-MC(2) with and without the observation window in the

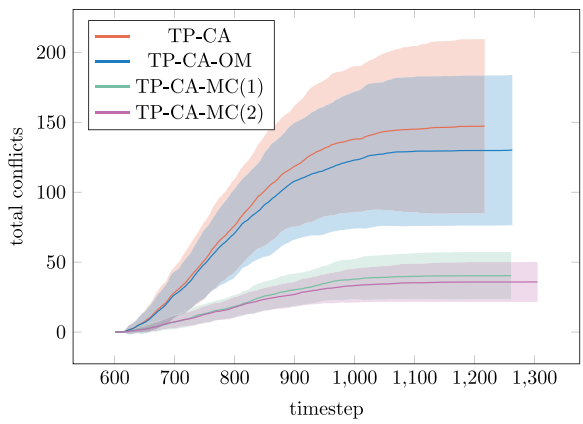


Fig. 11. Progression of the total number of conflicts in the unconstrained warehouse without observation window ($\alpha = 0.2$).

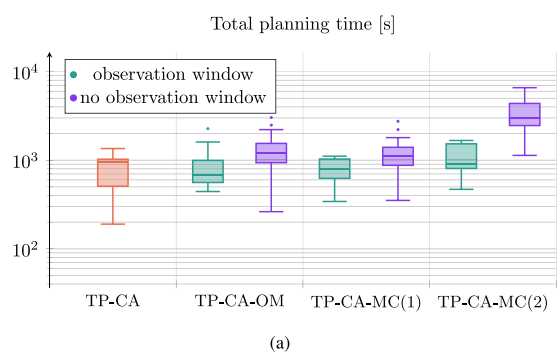


Fig. 12. Planning and replanning time comparison with and without the observation window for the unconstrained warehouse ($\alpha = 0.2$, Table A.12). The y-axis follows a log-scale and reports the seconds.

unconstrained warehouse environment. As one could expect, performance in terms of makespan and number of conflicts is worse with respect to the case in which the observation window is exploited. The employment of Markovian models (Fig. 10(a)) still leads to a strong decrease in terms of the number of conflicts when compared with TP-CA; however, the makespan (Fig. 10(b)) presents some increase (338 time steps for TP-CA, 481 time steps for TP-CA-MC(1), and 470 time steps for TP-CA-MC(2), on average). The occupancy model, instead, does not seem to be particularly effective in both metrics. Regarding the number of deadlocks (Fig. 10(c)), no clear trend can be identified. Without the observation window, the number of deadlocks is lower, comparable, or higher when using TP-CA-OM, TP-CA-MC(1), and TP-CA-MC(2), respectively, compared to their counterparts that utilize the observation window. Overall, the number of deadlocks remains similar to that of TP-CA.

Fig. 11 depicts the progression of the total number of conflicts over time. The x-axis represents the time steps, while the y-axis shows the cumulative number of conflicts observed up to each time step. As expected, as time progresses, the rate at which conflicts grow decreases, reflecting the knowledge gained from refining the model. This effect is particularly evident for Markovian models, which not only exhibit a substantially lower number of conflicts compared to TP-CA and TP-CA-OM, but also a small rate of change. This implies that Markovian models more accurately capture the behavior of external agents. The number of conflicts remains constant after 1100–1200 time steps as the number of agents operating within the environment decreases, since there are no tasks to assign.

Fig. 12 reports the time necessary for the algorithms to plan all paths. In the case of incrementally built models, the decrease in the

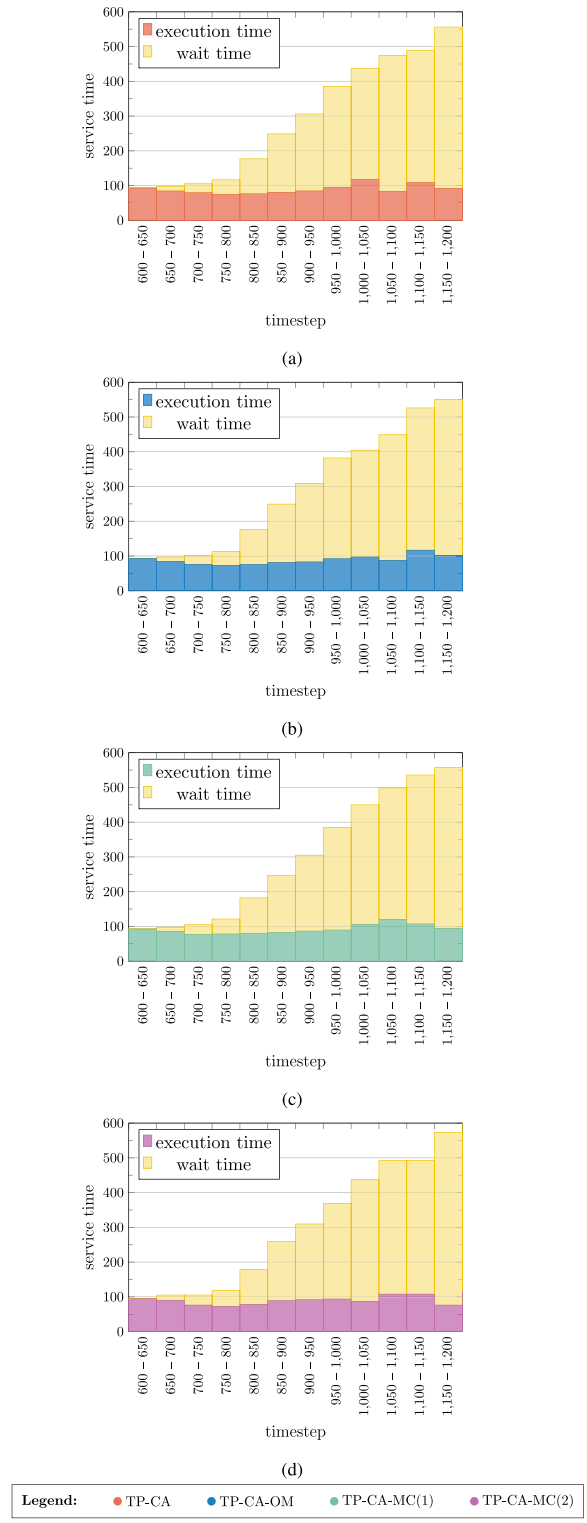


Fig. 13. Service time in the unconstrained warehouse without observation window ($\alpha = 0.2$).

number of conflicts is not enough to balance the additional time to consult the model. Thus, the increase of computational effort is noticeable for TP-CA-OM (1269 s) and TP-CA-MC(1) (1177 s) with respect to TP-CA (807 s). The increase is even more substantial for TP-CA-MC(2) (3692 s).

We present further results concerning the time required to complete tasks. Fig. 13 shows the service times for TP-CA, TP-CA-OM, TP-CA-MC(1), and TP-CA-MC(2), segmented by time step intervals. Service time measures the average time required to complete a task from its appearance in the task set. In this analysis, the wait time (the interval between a task's appearance and its assignment to a team agent) is depicted in yellow. As expected, wait time increases as time progresses. However, it is noticeable that the task execution time remains constant, implying that the increase in makespan that we have seen in Fig. 10(b) does not reflect too much on the time required to complete individual tasks. This can be explained by noting that, if the tasks are executed sequentially, the makespan is large, even if the service time (measuring how fast individual tasks are executed) remains low. With TP-CA-OM and TP-CA-MC, paths are chosen to proactively avoid collisions, which can result in team agents planning longer individual paths. Since team agents use the same model, they tend to cluster around the same (portions of) paths, progressing more sequentially than in parallel, leading to a higher overall makespan. In contrast, TP-CA, by just reacting to conflicts, produces sparser paths, allowing multiple tasks to proceed in parallel, at the cost of occasional delays due to conflicts.

One-way warehouse environment

For what concerns the warehouse environment with one-way corridors, as in the case of the observation window, we see that the employment of the model is more effective than in the unconstrained warehouse, thanks to the more structured environment that leads to a more predictable behavior.

Fig. 14 shows the comparison in terms of number of conflicts (Fig. 14(a)), makespan (Fig. 14(b)), and number of deadlocks (Fig. 14(c)) in the one-way constrained warehouse, considering TP-CA and the proposed models with and without the observation window. We see that results with and without the observation window are very similar (the observation window slightly provides more benefits). This shows that when the behavior of external agents follows a structured pattern (in this case imposed by the environment), our method is effective even without a prior observation phase.

For what concerns the evolution of conflicts over time reported in Fig. 15 and the service time reported in Fig. 16, the results follow the same pattern of the unconstrained warehouse environment. Our approach is able to decrease conflicts while keeping a stable execution time of tasks.

Fig. 17 reports the total planning and replanning time for this setting. Computational times for TP-CA-OM (625 s) and TP-CA-MC(1) (570 s) are almost identical to those with the observation window, and smaller than TP-CA (627 s). This shows that even without a prior observation phase, our approach deals well with the cost of consulting the model. TP-CA-MC(2) is computationally very expensive (1772 s), as in the previous case.

Overall, experimental results in this incremental observation setting align with the results observed in the scenario with the observation window, highlighting the effectiveness of our approach in structured environments.

10. Discussion

In this section, we discuss the effectiveness of the proposed approach under different conditions and we evaluate its limitations.

The structure of the environment plays a crucial role in determining the effectiveness of modeling external agents for planning. In constrained environments, our approach proves effective regardless of whether the model is provided beforehand (Fig. 8) or must be learned incrementally (Fig. 14). The structured nature of the external agent movements allows models to reduce conflicts, even when prior knowledge is unavailable (although some performance degradation occurs in this case).

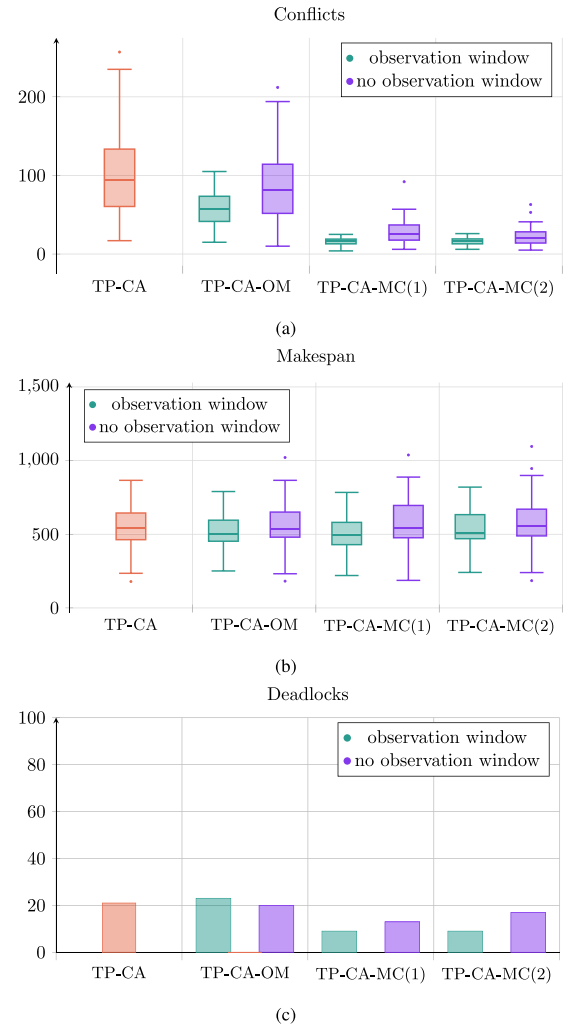


Fig. 14. Performance comparison of number of conflicts (Table A.13), makespan (Table A.14), and number of deadlocks (Table A.15) with and without the observation window for the warehouse with one-way constraints ($\alpha = 0.2$).

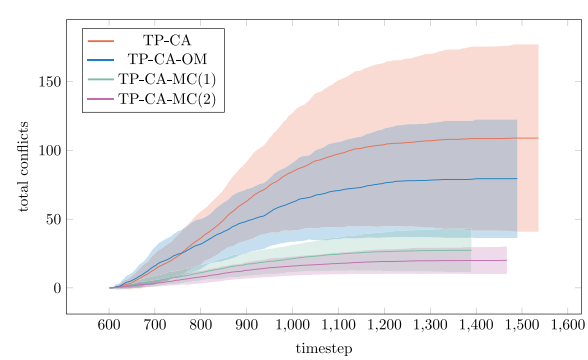


Fig. 15. Progression of conflicts in the warehouse with one-way constraints without observation window ($\alpha = 0.2$).

In environments with fewer constraints on their movements, external agents have more freedom to maneuver and the benefits of the proposed approach depend on the availability of a pre-learned model. When such a model is present (Fig. 6), our method reduces the number of conflicts, and for small values of α, the makespan and computational cost remain unchanged or increase slightly with respect

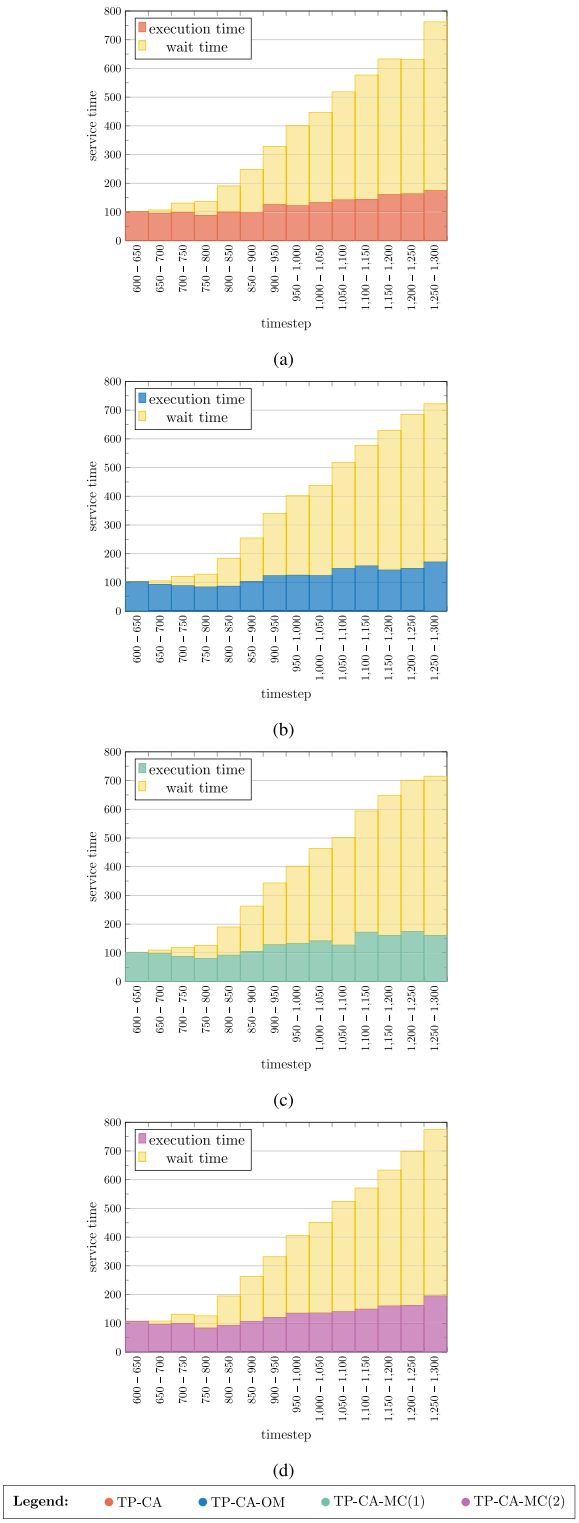


Fig. 16. Service time in the warehouse with one-way constraints without observation window ($\alpha = 0.2$).

to TP-CA. The most challenging scenario arises when the environment is unconstrained and the model is learned incrementally (Fig. 10, purple plot). In this case, our results indicate that Markovian models can still reduce the number of conflicts, but at the expense of higher makespan and computational cost, even for small values of α. Therefore, in such scenarios, a purely reactive approach could be preferable, as it avoids

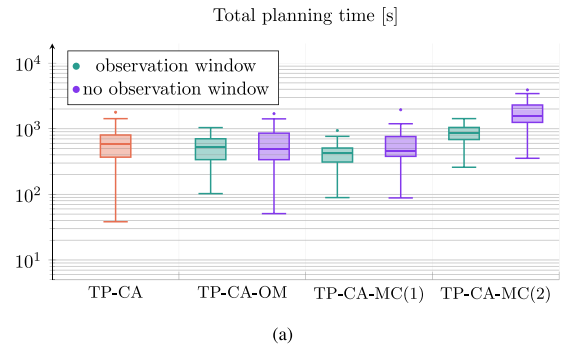


Fig. 17. Planning and replanning time comparison with and without the observation window for the warehouse with one-way constraints ($\alpha = 0.2$, Table A.16). The y-axis follows a log-scale and reports the seconds.

the overhead of maintaining a model of external agents that offers limited benefits.

The trade-off between computational time and solution quality depends not only on the complexity of the environment, but also on the choice of α and of the model. The parameter α controls the balance between path efficiency and collision avoidance. A low value of α prioritizes short paths with minimal deviations from the optimal ones, leading to faster planning times. However, this approach increases the likelihood of conflicts with external agents, requiring frequent replanning, which can significantly impact overall computational costs. Conversely, high values of α assign greater importance to avoiding external agents, reducing the need for reactive adjustments but increasing the complexity of the planning phase (and, typically, the length of the paths). As α grows, the length of the paths increases, requiring the planner to expand more nodes, thereby raising the computational time. Our experimental results indicate that small α values yield the most favorable trade-off, ensuring efficient task execution while maintaining an acceptable number of conflicts.

When comparing the models, Markovian models offer superior performance compared to the probabilistic occupancy model in reducing the number of conflicts, and similar performance in minimizing the makespan. This is due to their ability to account for the temporal dependencies in external agent behavior, allowing for more accurate predictions and better collision avoidance. By incorporating time-dependent edge weights, Markovian models can foresee the likelihood of conflicts with external agents over time, thus improving the overall efficiency of path planning. However, the benefits of Markovian models come with a trade-off in terms of computational cost. The added complexity of maintaining and updating these models during the planning process can substantially increase the computational burden. This is particularly true for second-order Markovian models, which involve a history composed of the current and past states of external agents. As the order of the Markov models increases, the number of variables that represent the history grows, leading to a higher computational cost due to the increased state space and the need to compute more complex transitions. Overall, the Markovian model of first order ($n = 1$) is able to reach a good balance between solution quality and computational cost. Probabilistic occupancy models, which assign fixed probabilities to agent locations based on historical data, are computationally simpler. They offer a reasonable estimate of collision likelihood without considering temporal relationships between external agent movements, making them less accurate in predicting future conflicts. While they are more efficient in terms of computational time, they do not provide much improvement to planning in dynamic environments, where agent behavior is time-dependent.

10.1. Differences between the models

To further clarify the differences between the models that inform planning in our approach, we present an illustrative example.

The environment shown in Fig. 18(a) is a small warehouse where the movement of external agents is constrained by one-way constraints along the horizontal aisles, as indicated by the arrows. Fig. 18(b) shows the probabilistic occupancy model as a heatmap, where darker locations denote a higher probability of occupancy according to the collected observations over a window of 180 time steps. This provides a snapshot of the static, probabilistic occupancy grid, where the estimate of the conflict likelihood is based on the historic frequency of occupancy. Fig. 18(c) shows the transition matrix of a first-order Markovian model, showing the most likely move an external agent is predicted to make at each location, constructed from the same observation samples used in Fig. 18(b). This model is used to predict the positions of external agents that evolve with time, based on their most recently known locations. Fig. 18(b) also shows that the model is successful at capturing the movement constraints along the aisles.

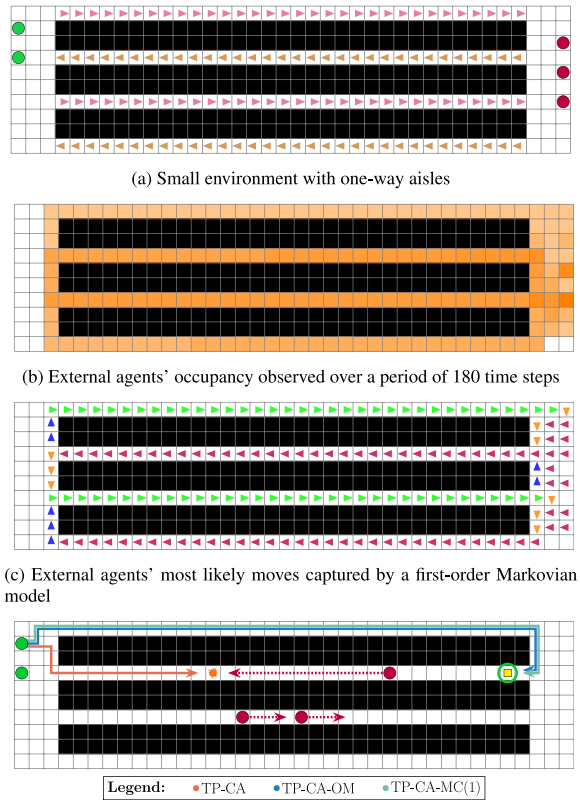
In Fig. 18(d), we present the differences in the paths to reach a pickup location, produced by TP-CA, TP-CA-OM, and TP-CA-MC(1), all with the same starting conditions. Solid green circles represent team agents, solid purple circles denote external agents, and the yellow square represents the pickup location of a task being executed. A value of $\alpha = 0.8$ is used for both TP-CA-OM and TP-CA-MC(1). The reactive approach of TP-CA minimizes path length not accounting for potential conflicts, resulting in rerouting after the presence of an external agent coming in the opposite direction is detected. In constrast, TP-CA-OM and TP-CA-MC(1) produce the same path, which is suboptimal in terms of path length, but safer in terms of the likelihood of conflicts, as it traverses the topmost horizontal aisle in the same direction enforced by the one-way constraint. It must be noted that the reason behind these two approaches producing the same path is substantially different. TP-CA-OM chooses to traverse the topmost aisle because it is generally less occupied, while TP-CA-MC(1) is choosing the most convenient path according to the most recently known external agents' locations and their predicted temporal evolution.

10.2. Scalability

We present here a quantitative assessment of the scalability of our modeling approaches. In particular, our aim is to demonstrate how the computational effort scales when increasing problem complexity. We present the total computational time required to plan for all tasks, including the additional planning effort for rerouting after a conflict. The objective is to provide empirical evidence supporting the theoretical complexity analysis presented in Section 7.5.2.

In an environment, problem complexity is mainly linked to three different dimensions: (i) the number of team agents, (ii) the number of tasks assigned to the team agents, and (iii) the number of external agents. We vary independently each quantity in a neighborhood of the value used in our previous experiments (see Table 1). The two warehouse environments in Fig. 5 are used and we generate 50 instances of the problem over which we take the average of our metric. Incremental model construction is considered, as this setting proved to be the most computationally intensive. The value of the parameter α is set to 0.2, consistent with our previous experimental settings.

Fig. 19 illustrates the effect of increasing the number of team agents on the planning time for TP-CA-OM, TP-CA-MC(1), and TP-CA-MC(2). TP-CA is also included for reference. The number of team agents that were considered are 4, 6, and 8. We observe that varying the number of team agents did not significantly affect the planning time in both environments. This suggests that the proposed approaches can scale efficiently with the number of team agents, and the computational overhead introduced by additional team agents is minimal. Consistently with Section 7.5.2, the computational complexity of consulting and



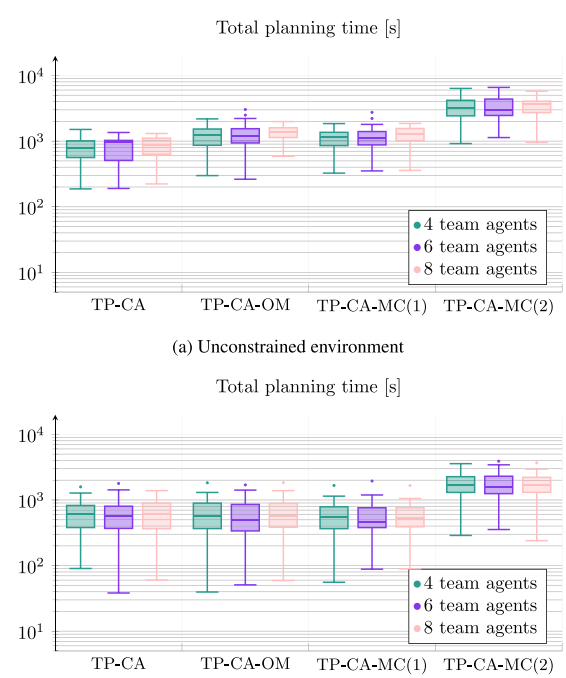
(a) Small environment with one-way aisles

(b) External agents' occupancy observed over a period of 180 time steps

(c) External agents' most likely moves captured by a first-order Markovian model

(d) Comparing paths across TP-CA, TP-CA-OM, and TP-CA-MC(1)

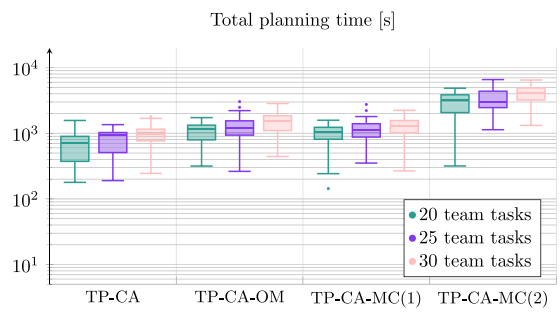
Fig. 18. An example to illustrate the differences between the proposed modeling approaches.



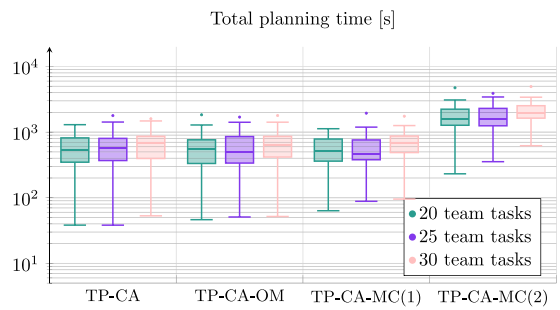
(a) Unconstrained environment

(b) Environment with one-way constraints

Fig. 19. Total planning time across a variable number of team agents ($\alpha = 0.2$) in the unconstrained warehouse environment (Fig. 19(a), Table A.17) and in the one-way warehouse environment (Fig. 19(b), Table A.18).

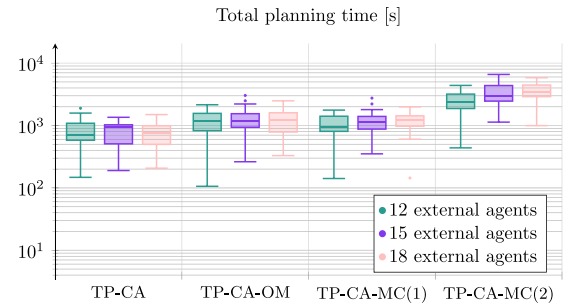


(a) Unconstrained environment

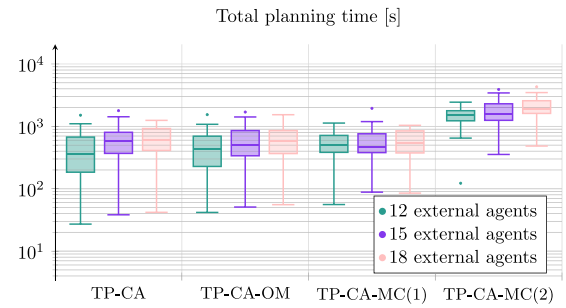


(b) Environment with one-way constraints

Fig. 20. Total planning time across a variable number of tasks for the team agents ($\alpha = 0.2$) in the unconstrained warehouse environment (Fig. 20(a), Table A.19) and in the one-way warehouse environment (Fig. 20(b), Table A.20).



(a) Unconstrained environment



(b) Environment with one-way constraints

Fig. 21. Total planning time across a variable number of external agents ($\alpha = 0.2$) in the unconstrained warehouse environment (Fig. 21(a), Table A.21) and in the one-way warehouse environment (Fig. 21(b), Table A.22).

updating the models scales primarily with the number of external agents and tasks, rather than with the number of team agents.

Fig. 20 reports the experimental results for a variable number of tasks for the team agents. The number of tasks that are considered are 20, 25, and 30. As expected, an increase in the number of tasks leads to a proportional increase in the total planning time. However, what is interesting to observe is that, in both environments, the additional tasks are handled without a disproportionate increase in planning time. This ascending trend is more accentuated in the unconstrained environment (Fig. 20(a)) but, overall, the results are compatible with a linear increment, as discussed in Section 7.5.2.

Fig. 21 illustrates the relationship between planning time and an increasing number of external agents. The numbers of external agents that are considered are 12, 15, and 18. In these settings, the rate of increment of the planning time is similar to what is observed for a variable number of tasks. This confirms that the complexity introduced by external agents – through their belief state updates and potential conflicts with team agents – scales linearly as expected (see Section 7.5.2).

A remark should be made on the effect of using higher-order Markovian models on the planning time. As expected, increasing the order of the Markov model results in a longer planning time. This increase is primarily due to the expanded state space, which requires more complex arithmetic operations (i.e., matrix multiplications) during both model consultation and belief update. This is evident for TP-CA-MC(2) in all the settings we consider.

11. Conclusion

This paper introduced a novel MAPD variant (MAPD-EA), which deals with MAPD problems in dynamic environments with external agents, and a solving approach for MAPD-EA. Our proposed method integrates models of external agents' behavior into the MAPD planning algorithm to proactively reduce conflicts, thereby improving the efficiency of task completion. We used both probabilistic occupancy models and Markovian models. The experimental results highlight that incorporating external agent modeling can substantially reduce conflicts and prevent deadlocks, especially when external agents follow structured movement patterns. The Markovian models proved effective in capturing external agents' behavior, allowing team agents to maintain lower conflict rates and consistent task execution times even without an initial observation period. The decrease in the number of replans is able to balance the additional computational time required for the consultation of the model.

Future work could explore adaptive model selection, i.e., dynamically choosing the most appropriate model or algorithm based on the current conditions, to further enhance performance across varied dynamic environments. Additionally, a promising direction could involve using deep learning models [40] that predict long-term agent trajectories or hybrid approaches that combine learned models with probabilistic reasoning [59]. Other interesting future research lines consist of considering multiple obstacle types and outdoor environments. The presence of different types of obstacles requires building distinct behavioral models and then integrating them into the team agents' planning process. Modeling external agents in outdoor environments also presents challenges, since movements are usually less constrained and more difficult to model, and assuming a complete view of the entire environment is often impractical. Thus, it may be necessary to build the model selectively, focusing only on specific areas of the environment that are most relevant to the team agents' tasks and interactions.

CRediT authorship contribution statement

Luca Bonalumi: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Conceptualization. **Benedetta Flammini:** Writing – review & editing, Writing – original

Table A.1
Number of conflicts (mean and standard deviation) in the unconstrained warehouse environment with observation window (Fig. 6(a)). Note that TP-CA corresponds to $\alpha = 0$.

α	TP-CA-OM	TP-CA-MC(1)	TP-CA-MC(2)
TP-CA		107.80 ± 49.54	
0.1	111.20 ± 40.99	44.80 ± 15.54	32.20 ± 14.48
0.2	100.20 ± 39.08	32.80 ± 11.34	24.00 ± 10.64
0.4	106.20 ± 34.12	34.80 ± 13.12	24.00 ± 4.52
0.6	109.40 ± 34.11	42.20 ± 12.80	24.80 ± 6.31
0.8	140.80 ± 78.37	43.00 ± 9.82	31.80 ± 15.69

Table A.2
Makespan (mean and standard deviation) in the unconstrained warehouse environment with observation window (Fig. 6(b)).

α	TP-CA-OM	TP-CA-MC(1)	TP-CA-MC(2)
TP-CA		338.00 ± 71.14	
0.1	358.20 ± 88.45	335.00 ± 69.00	351.60 ± 90.53
0.2	376.80 ± 98.56	350.00 ± 88.20	350.00 ± 89.46
0.4	379.80 ± 64.85	404.80 ± 88.57	362.00 ± 84.86
0.6	418.60 ± 132.25	420.40 ± 91.00	383.60 ± 102.79
0.8	489.00 ± 182.84	452.00 ± 122.32	461.60 ± 116.86

Table A.3
Number of deadlocks in the unconstrained warehouse environment with observation window (Fig. 6(c)).

α	TP-CA-OM	TP-CA-MC(1)	TP-CA-MC(2)
TP-CA		32	
0.1	45	33	32
0.2	46	31	22
0.4	47	29	30
0.6	40	32	30
0.8	40	37	32

draft, Visualization, Software, Methodology, Conceptualization. **Davide Azzalini:** Supervision, Methodology, Conceptualization. **Francesco Amigoni:** Writing – review & editing, Validation, Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

Luca Bonalumi is financially supported by Leonardo S.p.A and Benedetta Flammini is financially supported by the ABB – Politecnico di Milano Joint Research Center. This paper is supported by PNRR-PE-AI FAIR project funded by the NextGeneration EU program.

Appendix

See Tables A.1–A.22.

Data availability

Data will be made available on request.

Table A.4
Planning time (mean and standard deviation) in the unconstrained warehouse environment with observation window (Fig. 7).

α	TP-CA-OM	TP-CA-MC(1)	TP-CA-MC(2)
TP-CA		807.06 ± 409.89	
0.1	803.43 ± 470.40	793.90 ± 286.85	1061.63 ± 379.41
0.2	944.53 ± 570.99	792.91 ± 253.12	1077.25 ± 402.90
0.4	1392.66 ± 744.51	1136.12 ± 435.24	1655.29 ± 472.57
0.6	2802.66 ± 1709.25	1978.72 ± 799.79	2452.35 ± 756.33
0.8	7751.61 ± 6195.94	4716.79 ± 1294.85	7416.35 ± 3623.73

Table A.5
Number of conflicts (mean and standard deviation) in the one-way warehouse environment with observation window (Fig. 8(a)).

α	TP-CA-OM	TP-CA-MC(1)	TP-CA-MC(2)
TP-CA		106.19 ± 60.77	
0.1	62.12 ± 27.45	23.62 ± 9.55	22.04 ± 14.94
0.2	61.88 ± 31.73	16.35 ± 7.87	15.15 ± 6.46
0.4	58.31 ± 26.26	13.65 ± 6.70	12.00 ± 6.16
0.6	61.58 ± 37.63	13.00 ± 6.88	10.54 ± 5.64
0.8	70.96 ± 56.30	12.42 ± 6.67	12.54 ± 7.95

Table A.6
Makespan (mean and standard deviation) in the one-way warehouse environment with observation window (Fig. 8(b)).

α	TP-CA-OM	TP-CA-MC(1)	TP-CA-MC(2)
TP-CA		527.69 ± 153.32	
0.1	508.77 ± 138.99	494.65 ± 138.71	491.27 ± 144.88
0.2	524.38 ± 144.66	512.50 ± 149.33	507.73 ± 136.73
0.4	540.08 ± 145.15	535.58 ± 146.74	542.65 ± 141.88
0.6	582.69 ± 173.97	591.31 ± 182.39	575.69 ± 159.67
0.8	678.00 ± 227.81	691.00 ± 219.08	670.38 ± 206.34

Table A.7
Number of deadlocks in the one-way warehouse environment with observation window (Fig. 8(c)).

α	TP-CA-OM	TP-CA-MC(1)	TP-CA-MC(2)
TP-CA		21	
0.1	21	8	14
0.2	23	9	9
0.4	18	13	15
0.6	22	14	9
0.8	17	7	8

Table A.8
Planning time (mean and standard deviation) in the one-way warehouse environment with observation window (Fig. 9).

α	TP-CA-OM	TP-CA-MC(1)	TP-CA-MC(2)
TP-CA		627.80 ± 381.18	
0.1	496.31 ± 306.84	415.29 ± 185.31	858.50 ± 528.09
0.2	609.43 ± 428.91	434.51 ± 197.84	818.75 ± 361.14
0.4	777.41 ± 464.53	583.28 ± 267.40	1082.24 ± 511.30
0.6	1296.03 ± 1046.25	932.30 ± 419.92	1628.44 ± 827.47
0.8	2859.39 ± 2324.69	1942.34 ± 961.67	4440.76 ± 2785.86

Table A.9
Number of conflicts in the unconstrained warehouse environment with and without observation window with $\alpha = 0.2$ (Fig. 10(a)).

TP-CA	107.80 ± 49.54
TP-CA-OM with obs. window	100.20 ± 39.08
TP-CA-OM without obs. window	135.70 ± 49.76
TP-CA-MC(1) with obs. window	32.80 ± 11.34
TP-CA-MC(1) without obs. window	42.76 ± 15.98
TP-CA-MC(2) with obs. window	24.00 ± 10.64
TP-CA-MC(2) without obs. window	38.09 ± 13.50

Table A.10
Makespan in the unconstrained warehouse environment with and without observation window with $\alpha = 0.2$ (Fig. 10(b)).

TP-CA	338.00 ± 71.14
TP-CA-OM with obs. window	376.80 ± 98.56
TP-CA-OM without obs. window	453.45 ± 105.32
TP-CA-MC(1) with obs. window	350.00 ± 88.20
TP-CA-MC(1) without obs. window	481.27 ± 115.30
TP-CA-MC(2) with obs. window	350.00 ± 89.46
TP-CA-MC(2) without obs. window	470.36 ± 116.79

Table A.11
Number of deadlocks in the unconstrained warehouse environment with and without observation window with $\alpha = 0.2$ (Fig. 10(c)).

TP-CA	32
TP-CA-OM with obs. window	46
TP-CA-OM without obs. window	26
TP-CA-MC(1) with obs. window	31
TP-CA-MC(1) without obs. window	30
TP-CA-MC(2) with obs. window	22
TP-CA-MC(2) without obs. window	31

Table A.12
Planning time in the unconstrained warehouse environment with and without observation window with $\alpha = 0.2$ (Fig. 12).

TP-CA	807.06 ± 409.89
TP-CA-OM with obs. window	944.53 ± 570.99
TP-CA-OM without obs. window	1269.70 ± 578.73
TP-CA-MC(1) with obs. window	792.91 ± 253.12
TP-CA-MC(1) without obs. window	1177.96 ± 478.53
TP-CA-MC(2) with obs. window	1077.25 ± 402.90
TP-CA-MC(2) without obs. window	3372.86 ± 1317.66

Table A.13
Number of conflicts in the one-way warehouse environment with and without observation window with $\alpha = 0.2$ (Fig. 14(a)).

TP-CA	106.19 ± 60.77
TP-CA-OM with obs. window	61.88 ± 31.73
TP-CA-OM without obs. window	86.05 ± 46.07
TP-CA-MC(1) with obs. window	16.35 ± 7.87
TP-CA-MC(1) without obs. window	29.17 ± 15.40
TP-CA-MC(2) with obs. window	15.15 ± 6.46
TP-CA-MC(2) without obs. window	22.22 ± 11.25

Table A.14
Makespan in the one-way warehouse environment with and without observation window with $\alpha = 0.2$ (Fig. 14(b)).

TP-CA	527.69 ± 153.32
TP-CA-OM with obs. window	524.38 ± 144.66
TP-CA-OM without obs. window	554.45 ± 151.53
TP-CA-MC(1) with obs. window	512.50 ± 149.33
TP-CA-MC(1) without obs. window	565.37 ± 161.92
TP-CA-MC(2) with obs. window	507.73 ± 136.73
TP-CA-MC(2) without obs. window	579.97 ± 165.05

Table A.15
Number of deadlocks in the one-way warehouse environment with and without observation window with $\alpha = 0.2$ (Fig. 14(c)).

TP-CA	21
TP-CA-OM with obs. window	23
TP-CA-OM without obs. window	20
TP-CA-MC(1) with obs. window	9
TP-CA-MC(1) without obs. window	13
TP-CA-MC(2) with obs. window	9
TP-CA-MC(2) without obs. window	17

Table A.16
Planning time in the one-way warehouse environment with and without observation window with $\alpha = 0.2$ (Fig. 17).

TP-CA	627.80 ± 381.18
TP-CA-OM with obs. window	609.43 ± 428.91
TP-CA-OM without obs. window	625.98 ± 391.64
TP-CA-MC(1) with obs. window	434.51 ± 197.84
TP-CA-MC(1) without obs. window	570.42 ± 334.26
TP-CA-MC(2) with obs. window	818.75 ± 361.14
TP-CA-MC(2) without obs. window	1772.40 ± 792.80

Table A.17
Total planning time across a variable number of team agents in the unconstrained warehouse environment ($\alpha = 0.2$, Fig. 19(a)).

	4 team agents	6 team agents	8 team agents
TP-CA	784.36 ± 329.81	807.06 ± 409.89	855.70 ± 288.30
TP-CA-OM	1192.81 ± 443.66	1269.70 ± 578.73	1356.57 ± 298.96
TP-CA-MC(1)	1081.73 ± 358.68	1177.96 ± 478.53	1238.83 ± 397.02
TP-CA-MC(2)	3268.26 ± 1378.12	3372.86 ± 1317.66	3457.12 ± 1312.31

Table A.18
Total planning time across a variable number of team agents in the one-way warehouse environment ($\alpha = 0.2$, Fig. 19(b)).

	4 team agents	6 team agents	8 team agents
TP-CA	639.50 ± 331.31	627.80 ± 381.18	651.27 ± 342.19
TP-CA-OM	625.09 ± 371.62	625.98 ± 391.64	643.73 ± 373.20
TP-CA-MC(1)	579.73 ± 321.52	570.42 ± 334.26	577.31 ± 299.84
TP-CA-MC(2)	1791.25 ± 749.94	1772.40 ± 792.80	1770.79 ± 673.23

Table A.19
Total planning time across a variable number of tasks for the team agents in the unconstrained warehouse environment ($\alpha = 0.2$, Fig. 20(a)).

	20 team tasks	25 team tasks	30 team tasks
TP-CA	709.91 ± 339.12	807.06 ± 409.89	963.88 ± 389.82
TP-CA-OM	1079.29 ± 409.58	1269.70 ± 578.73	1482.07 ± 489.89
TP-CA-MC(1)	987.54 ± 356.12	1177.96 ± 478.53	1296.90 ± 469.64
TP-CA-MC(2)	2863.11 ± 1280.18	3372.86 ± 1317.66	4070.40 ± 1248.98

Table A.20
Total planning time across a variable number of tasks for the team agents in the one-way warehouse environment (Fig. 20(b)).

	20 team tasks	25 team tasks	30 team tasks
TP-CA	582.78 ± 332.64	627.80 ± 381.18	670.41 ± 368.35
TP-CA-OM	589.70 ± 362.22	625.98 ± 391.64	672.71 ± 373.10
TP-CA-MC(1)	567.27 ± 292.68	570.42 ± 334.26	707.71 ± 323.25
TP-CA-MC(2)	1761.36 ± 754.52	1772.40 ± 792.80	2126.78 ± 723.60

Table A.21
Total planning time across a variable number of external agents in the unconstrained warehouse environment (Fig. 21(a)).

	4 ext. agents	6 ext. agents	8 ext. agents
TP-CA	834.16 ± 420.81	807.06 ± 409.89	805.39 ± 357.26
TP-CA-OM	1211.47 ± 533.61	1269.70 ± 578.73	1291.32 ± 576.86
TP-CA-MC(1)	1057.39 ± 424.00	1177.96 ± 478.53	1230.59 ± 390.32
TP-CA-MC(2)	2445.97 ± 910.91	3372.86 ± 1317.66	3616.03 ± 1081.55

Table A.22
Total planning time across a variable number of external agents in the one-way warehouse environment (Fig. 21(b)).

	4 ext. agents	6 ext. agents	8 ext. agents
TP-CA	442.03 ± 346.41	627.80 ± 381.18	638.88 ± 334.03
TP-CA-OM	492.69 ± 333.29	625.98 ± 391.64	628.52 ± 352.59
TP-CA-MC(1)	548.76 ± 237.25	570.42 ± 334.26	574.44 ± 252.61
TP-CA-MC(2)	1490.04 ± 483.70	1772.40 ± 792.80	2105.40 ± 741.31

References

[1] J. Yu, S. LaValle, Optimal multirobot path planning on graphs: Complete algorithms and effective heuristics, IEEE Trans. Robot. 32 (5) (2016) 1163–1177.
[2] H. Ma, J. Li, T. Kumar, S. Koenig, Lifelong multi-agent path finding for online pickup and delivery tasks, in: Proc. AAMAS, 2017, pp. 837–845.
[3] P. Wurman, R. D'Andrea, M. Mountz, Coordinating hundreds of cooperative, autonomous vehicles in warehouses, AI Mag. 29 (2008) 9–20.
[4] R. Morris, C. Pasareanu, K. Luckow, W. Malik, H. Ma, T. Kumar, S. Koenig, Planning, scheduling and monitoring for airport surface operations, in: Proc. AAAI Workshop on Planning for Hybrid Systems, 2016, pp. 608–614.
[5] M. Veloso, J. Biswas, B. Coltin, S. Rosenthal, Cobots: Robust symbiotic autonomous mobile service robots, in: Proc. IJCAI, 2015, pp. 4423–4429.
[6] D. Silver, Cooperative pathfinding, in: Proc. AIIDE, 2005, pp. 117–122.
[7] H. Ma, Graph-based multi-robot path finding and planning, Curr. Robot. Rep. 3 (3) (2022) 77–84.
[8] B. Flammini, D. Azzalini, F. Amigoni, Multi-agent pickup and delivery in presence of another team of robots (extended abstract), in: Proc. AAMAS, 2023, pp. 2562–2564.
[9] R. Stern, N. Sturtevant, A. Felner, S. Koenig, H. Ma, T. Walker, J. Li, D. Atzmon, L. Cohen, T. Kumar, R. Barták, E. Boyarski, Multi-agent pathfinding: Definitions, variants, and benchmarks, in: Proc. SOCS, 2019, pp. 151–158.
[10] J. Svancara, M. Vlk, R. Stern, D. Atzmon, R. Barták, Online multi-agent pathfinding, in: Proc. AAAI, 2019, pp. 7732–7739.
[11] J. Yu, S. LaValle, Structure and intractability of optimal multi-robot path planning on graphs, in: Proc. AAAI, 2013, pp. 1443–1449.
[12] P. Surynek, An optimization variant of multi-robot path planning is intractable, in: Proc. AAAI, 2010, pp. 1261–1263.
[13] J. Li, P. Surynek, A. Felner, H. Ma, T. Kumar, S. Koenig, Multi-agent path finding for large agents, in: Proc. AAAI, 2019, pp. 7627–7634.
[14] P. Surynek, Multi-agent path finding with continuous time and geometric agents viewed through satisfiability modulo theories (SMT), in: Proc. SOCS, 2019, pp. 200–201.
[15] D. Atzmon, Y. Zax, E. Kivity, L. Avitan, J. Morag, A. Felner, Generalizing multi-agent path finding for heterogeneous agents, in: Proc. SOCS, 2020, pp. 101–105.
[16] T. Walker, N. Sturtevant, A. Felner, Extended increasing cost tree search for non-unit cost domains, in: Proc. IJCAI, 2018, pp. 534–540.
[17] K. Yakovlev, A. Andreychuk, Any-angle pathfinding for multiple agents based on SIPP algorithm, in: Proc. ICAPS, 2017, pp. 586–593.
[18] W. Hönig, T. Kumar, L. Cohen, H. Ma, H. Xu, N. Ayanian, S. Koenig, Multi-agent path finding with kinematic constraints, in: Proc. ICAPS, 2016, pp. 477–485.
[19] A. Andreychuk, K. Yakovlev, P. Surynek, D. Atzmon, R. Stern, Multi-agent pathfinding with continuous time, Artificial Intelligence 305 (2022).
[20] H. Ma, W. Hönig, T. Kumar, N. Ayanian, S. Koenig, Lifelong path planning with kinematic constraints for multi-agent pickup and delivery, in: Proc. AAAI, 2019, pp. 7651–7658.
[21] T. Yamauchi, Y. Miyashita, T. Sugawara, Path and action planning in non-uniform environments for multi-agent pickup and delivery tasks, in: Proc. EUMAS, 2021, pp. 37–54.
[22] F. Kudo, K. Cai, A TSP-based online algorithm for multi-task multi-agent pickup and delivery, IEEE Robot. Autom. Lett. (2023).
[23] Z. Chen, J. Alonso-Mora, X. Bai, D. Harabor, P. Stuckey, Integrated task assignment and path planning for capacitated multi-agent pickup and delivery, IEEE Robot. Autom. Lett. 6 (2021) 5816–5823.
[24] Y. Li, H. Ye, R. Huang, H. Huang, H. Du, Multi-load agent path finding for online pickup and delivery problem, in: Proc. COCOON, 2023, pp. 285–296.
[25] Q. Xu, J. Li, S. Koenig, H. Ma, Multi-goal multi-agent pickup and delivery, in: Proc. IROS, 2022, pp. 9964–9971.
[26] A. Di Pietro, N. Basilico, F. Amigoni, Multi-agent pickup and delivery with task probability distribution, in: Proc. AAMAS, 2023, pp. 2580–2582.
[27] G. Lodigiani, N. Basilico, F. Amigoni, Robust multi-agent pickup and delivery with delays, in: Proc. ECMR, 2023, pp. 1–8.
[28] T. Yamauchi, Y. Miyashita, T. Sugawara, Standby-based deadlock avoidance method for multi-agent pickup and delivery tasks, in: Proc. AAMAS, 2022, pp. 1427–1435.
[29] Y. Fujitani, T. Yamauchi, Y. Miyashita, T. Sugawara, Deadlock-free method for multi-agent pickup and delivery problem using priority inheritance with temporary priority, in: Proc. KES, 2022, pp. 1552–1561.
[30] K. Okumura, M. Machida, X. Défago, Y. Tamura, Priority inheritance with backtracking for iterative multi-agent path finding, Artificial Intelligence 310 (2022) 1–22.
[31] B. Flammini, D. Azzalini, F. Amigoni, Preventing deadlocks for multi-agent pickup and delivery in dynamic environments, in: Proc. AAMAS, 2024, pp. 580–588.
[32] J.-M. Alkazzi, K. Okamura, A comprehensive review on leveraging machine learning for multi-agent path finding, IEEE Access 12 (2024) 57390–57409.
[33] G. Sartoretti, J. Kerr, Y. Shi, G. Wagner, T. Kumar, S. Koenig, H. Choset, Primal: Pathfinding via reinforcement and imitation multi-agent learning, IEEE Robot. Autom. Lett. 4 (3) (2019) 2378–2385.
[34] Z. Ma, Y. Luo, H. Ma, Distributed heuristic multi-agent path finding with communication, in: Proc. ICRA, 2021, pp. 8699–8705.
[35] W. Li, H. Chen, B. Jin, W. Tan, H. Zha, X. Wang, Multi-agent path finding with prioritized communication learning, in: Proc. ICRA, 2022, pp. 10695–10701.
[36] A. Skrynnik, A. Andreychuk, K. Yakovlev, A. Panov, When to switch: planning and learning for partially observable multi-agent pathfinding, IEEE Trans. Neural Netw. Learn. Syst. (2023).
[37] Q. Li, F. Gama, A. Ribeiro, A. Prorok, Graph neural networks for decentralized multi-robot path planning, in: Proc. IROS, 2020, pp. 11785–11792.
[38] Q. Li, W. Lin, Z. Liu, A. Prorok, Message-aware graph attention networks for large-scale multi-robot path planning, IEEE Robot. Autom. Lett. 6 (3) (2021) 5533–5540.
[39] M. Damani, Z. Luo, E. Wenzel, G. Sartoretti, PRIMAL2: Pathfinding via reinforcement and imitation multi-agent learning-lifelong, IEEE Robot. Autom. Lett. 6 (2) (2021) 2666–2673.
[40] A. Skrynnik, A. Andreychuk, M. Nesterova, K. Yakovlev, A. Panov, Learn to follow: Decentralized lifelong multi-agent pathfinding via planning and learning, in: Proc. AAAI, 2024, pp. 17541–17549.
[41] S. Albrecht, P. Stone, Autonomous agents modelling other agents: A comprehensive survey and open problems, Artificial Intelligence 258 (2018) 66–95.
[42] F. Schadd, S. Bakkes, P. Spronck, Opponent modeling in real-time strategy games, in: Proc. GAME-on, 2007, pp. 61–70.
[43] K. Laviers, G. Sukthankar, D. Aha, M. Molineaux, Improving offensive performance through opponent modeling, in: Proc. AIIDE, 2009, pp. 58–63.
[44] H. Kautz, J. Allen, Generalized plan recognition, in: Proc. AAAI, 1986, pp. 32–37.
[45] D. Avrahami-Zilberbrand, G. Kaminka, H. Zarosim, Fast and complete symbolic plan recognition: Allowing for duration, interleaved execution, and lossy observations, in: Proc. AAAI Workshop on Modeling Others from Observations, 2005, pp. 25–19.
[46] M. Ramírez, H. Geffner, Plan recognition as planning, in: Proc. IJCAI, 2009, pp. 1778–1783.
[47] G. Brown, Iterative solution of games by fictitious play, Act. Anal. Prod. Alloc. 13 (1951) 374–376.
[48] N. Agmon, S. Kraus, G. Kaminka, Multi-robot perimeter patrol in adversarial settings, in: Proc. ICRA, 2008, pp. 2339–2345.
[49] J. Bruce, M. Bowling, B. Browning, M. Veloso, Multi-robot team response to a multi-robot opponent team, in: Proc. ICRA, 2003, pp. 2281–2286.
[50] M. Cognetti, G. Oriolo, P. Peliti, L. Rosa, P. Stegagno, Cooperative control of a heterogeneous multi-robot system based on relative localization, in: Proc. IROS, 2014, pp. 350–356.
[51] H. Nguyen, M. Chesser, L. Koh, H. Rezatofighi, D. Ranasinghe, TrackerBots: Autonomous unmanned aerial vehicle for real-time localization and tracking of multiple radio-tagged animals, J. Field Robot. 36 (3) (2019) 617–635.
[52] Y. Cui, J. Chen, H. Lin, Z. Shu, T. Huang, Cooperative multi-AAV path planning for discovering and tracking multiple radio-tagged targets, IEEE Trans. Syst. Man Cybern.: Syst. (2025) 1–13.
[53] F. Chen, H. Nguyen, D. Taggart, K. Falkner, H. Rezatofighi, D. Ranasinghe, ConservationBots: Autonomous aerial robot for fast robust wildlife tracking in complex terrains, J. Field Robot. 41 (2) (2024) 443–469.
[54] T. Vintr, S. Molina, R. Senanayake, G. Broughton, Z. Yan, J. Ulrich, T. Kucner, C.S. Swaminathan, F. Majer, M. Stachová, A. Lilienthal, T. Krajnik, Time-varying pedestrian flow models for service robots, in: Proc. ECMR, 2019, pp. 1–7.
[55] T. Krajník, T. Vintr, S. Molina, J. Fentanes, G. Cielniak, O. Mozos, G. Broughton, T. Duckett, Warped hypertime representations for long-term autonomy of mobile robots, IEEE Robot. Autom. Lett. 4 (2019) 3310–3317.
[56] L. Sun, Z. Yan, S. Mellado, M. Hanheide, T. Duckett, 3DOF pedestrian trajectory prediction learned from long-term autonomous mobile robot deployment data, in: Proc. ICRA, 2018, pp. 5942–5948.
[57] T. Krajník, J. Fentanes, J. Santos, T. Duckett, Fremen: Frequency map enhancement for long-term mobile robot autonomy in changing environments, IEEE Trans. Robot. 33 (4) (2017) 964–977.
[58] J. Fentanes, B. Lacerda, T. Krajník, N. Hawes, M. Hanheide, Now or later? predicting and maximising success of navigation actions from long-term experience, in: Proc. ICRA, 2015, pp. 1112–1117.
[59] B. Lacerda, F. Faruq, D. Parker, N. Hawes, Probabilistic planning with formal performance guarantees for mobile service robots, Int. J. Robot. Res. 38 (9) (2019) 1098–1123.
[60] W. Budiharto, A. Santoso, D. Purwanto, A. Jazidie, Multiple moving obstacles avoidance of service robot using stereo vision, Telkomnika 9 (2011) 433–444.
[61] J. Kim, Y. Do, Moving obstacle avoidance of a mobile robot using a single camera, Procedia Eng. 41 (2012) 911–916.
[62] C. Fulgenzi, A. Spalanzani, C. Laugier, Dynamic obstacle avoidance in uncertain environment combining PVOs and occupancy grid, in: Proc. ICRA, 2007, pp. 1610–1616.
[63] O. Ibe, Markov Processes for Stochastic Modeling, Newnes, 2013.

Luca Bonalumi was born in Segrate (MI), Italy, in 1994. He got his Master of Science degree in Computer Engineering from the Politecnico di Milano in 2023 with a thesis on modeling agents' behavior in a Multi-Agent Pickup and Delivery (MAPD) problem. Currently, he is a Ph.D. student in Information and Technology at the Politecnico di Milano in the field of Artificial Intelligence with applications in aeronautics. His primary research involves collaborating with Leonardo S.p.A. on the flight control law clearance problem, where he is developing AI techniques to support the assessment of flight control law compliance.

Benedetta Flammini was born in Teramo (TE), Italy, in 1997. She received a Master of Science degree in Applied Data Science from Università degli Studi dell'Aquila in 2021 with a thesis on Anomaly Detection in Temporal Graphs using a Reinforcement Learning approach. Currently, she is a Ph.D. student in Data Analytics and Decision Sciences at Politecnico di Milano in the field of Artificial Intelligence and Robotics. Her main research interests are in Multi-Agent Coordination and Planning. In particular, her work is focused on Multi-Agent Pickup and Delivery (MAPD), proposing novel variants to bridge the gap with real-world scenarios and designing solution algorithms.

Davide Azzalini was born in Sondrio (SO), Italy, in 1993. He got a Master of Science degree in Computer Science and Engineering from the Politecnico di Milano in 2017. He got is Ph.D. at Politecnico di Milano in 2022, Dipartimento di Elettronica, Informazione e Bioingegneria. His research focuses on anomaly detection and fault detection both on autonomous robots and on time series in general. He has published in international scientific journals and conferences on artificial intelligence and robotics (e.g., AAMAS, IEEE ICRA, IEEE RA-L, IEEE PAMI).

Francesco Amigoni was born in Soresina (CR), Italy, on December 2, 1971. He got the Laurea degree in Computer Engineering from the Politecnico di Milano in 1996 and the Ph.D. degree in Computer Engineering and Automatica from the Politecnico di Milano in 2000. From December 1999 to September 2000, he was a visiting scholar at the Computer Science Department of Stanford University (USA). He has been an assistant professor (2002–2007) and an associate professor (2007–2021) and, since 2021, a full professor at the Dipartimento di Elettronica e Informazione of the Politecnico di Milano. His main research interests include agents and multiagent systems, autonomous mobile robotics, and the philosophical aspects of artificial intelligence.